

Evaluating Probabilistic Logic Factored MDPs for Behavior Selection in Self-Driving Cars

Héctor Avilés¹, Verónica Rodríguez², Alberto Reyes³,
Rubén Machucho¹, Marco Negrete⁵, Gabriel Ramirez⁴,
Karely Rivera¹, Myriam Pequeño¹, Ingridh Gracia¹,
Nicolás Luévano¹

Abstract

Probabilistic logic factored Markov decision processes (*PL-fMDPs*) have been recently proposed to generate action policies for behavior selection in autonomous vehicles with promising results. However, a thorough and comprehensive assessment of their performance is still required. We present an extensive evaluation of the effectiveness of PL-fMDPs in modeling behavior selection for self-driving cars. We carried out three main tests in order to evaluate: (a) the efficiency and effectiveness of PL-fMDPs with factored state transition functions learned from data obtained via the autonomous operation of a simulated self-driving vehicle, (b) robustness of PL-fMDPs to distribution shifts by contrasting the action policies obtained in the first test with driving decisions collected from humans, and (c) the effectiveness of action policies to drive a simulated self-driving car in more challenging road conditions. Through these tests, PL-fMDPs are compared with six well-established supervised learning models applied as behavior selectors. The results in terms of F1 score, precision-recall and training and testing time show the competitive edge of the PL-fMDPs with respect to the influential methods with which they were compared.

Keywords: Self-driving cars, Factored Markov decision processes, Probabilistic logic, Decision-making, Machine learning.

1 Introduction

Self-driving cars technology has made significant strides in recent years. Despite promising advances, different challenges must be addressed to transition from their current partial autonomy to fully automated operation (Chougule et al, 2024). A key capability is behavior selection, which involves deciding what reactive short-term maneuver or action (such as short stopping, cruising, lane changing, or braking) the

vehicle should take based on the current state of the environment (Da Lio et al, 2023; Badue et al, 2021; Tampus et al, 2020).

Behavior selection can be stated as mapping the observed state of the environment to a driving action choice for appropriate road operation. As a result, a variety of artificial intelligence techniques, ranging from rule-based and tree-based models to probabilistic and connectionist approaches, have been proposed (Hu et al, 2025; Chen et al, 2024; Ghraizi et al, 2023; Aksjonov and Kyrki, 2022). However, some of these models either do not handle uncertainty explicitly or have inference internals not sufficiently interpretable (Şensoy et al, 2025; Niu et al, 2025; Carvalho et al, 2019). Uncertainty is inherent in driving and handle it can improve safety. Model transparency, on the other hand, improves reliability on action selection. Probabilistic logic Markov decision processes (*PL-fMDPs*) (Bueno et al, 2016) have been recently applied to address these needs (Avilés et al, 2022, 2024). PL-fMDPs are factored Markov decision processes encoded as a probabilistic logic program (Riguzzi, 2023). In this form, simple probabilistic-logic rules can express intricate spatial relationships between participants on the road, as well as rewards for specific combinations of state-action pairs, while accounting for the probable effects of actions on the state of the environment. The solution of a PL-fMDP is a *reward-optimal action policy* in the form of *if-then* rules associating every possible state of the environment with a suitable action for each state. In addition, probabilistic logic paradigm provides with sophisticated inference procedures to efficiently solve probabilistic queries on a wide variety of probability distributions (Eiter et al, 2024; Derkinderen et al, 2024; Chavira and Darwiche, 2008). Although offering diverse strengths, the performance of PL-fMDPs has neither been thoroughly analyzed for behavior selection nor compared to other machine learning methods.

Therefore, in this paper we present an extensive evaluation of the effectiveness of PL-fMDPs to model behavior selection for self-driving cars. We perform three main tests to assess their performance. In Test 1, we evaluate PL-fMDPs with factored state transition functions learned from data collected from the autonomous operation of a simulated self-driving vehicle in a race-like scenario (see Figure 1). The scenario considers a one-way road with two lanes with up to 10 obstacle vehicles distributed over the entire road. The state of the environment is represented through seven binary variables. In addition, the self-driving car can perform four driving actions. This is an in-distribution test because both the testing and training data comes from the same generation data process, which is the autonomous operation of our self-driving vehicle using a previous version of PL-fMDPs as the decision-making technique (Avilés et al, 2024). In contrast, in Test 2 we assess the agreement of the action policies obtained in Test 1 but with examples gathered from human operating the self-driving car. The aim is to evaluate how sensitive PL-fMDPs are to distribution shifts. A distribution shift occurs when the frequencies of two frequency distributions of the same variables differ. Test 3 evaluates the effectiveness of action policies to drive the simulated self-driving car in unknown road conditions, which consider: (i) passing a slow vehicle obstructing the road with fast-moving vehicles approaching from behind (ii) an obstacle vehicle traveling in the opposite direction, and (iii) a moving obstacle vehicle failing to yield the right-of-way in an intersection. Since the speed and direction



Fig. 1: Race scenario that serves as testbed in our study. The self-driving car is the bright red one that is changing lane (from left to right).

of obstacle vehicles differ from those used in constructing PL-fMDPs, we consider this test involves significant environmental changes, leading to an out-of-distribution test. Through these tests, PL-fMDPs are compared with respect to F1 score, precision-recall and training and testing time with classification and regression trees (*CART*), extreme gradient boosting (*XGBoost*), random forests (*RFs*), naive Bayes (*NB*), multi-layer perceptron (*MLP*), and multinomial logistic regression (*LR*) applied to behavior selection. These methods were selected because their underlying foundations are well-understood, possess a widespread application in a variety of label prediction problems, have a great reputation, and cover diverse approaches to supervised learning. The results show the competitiveness of PL-fMDPs with respect to all of these powerful techniques.

The contribution of this work is threefold. First, this is the first time a comparative study between PL-fMDPs and a wide variety of machine learning models is presented. Second, it is shown that PL-fMDPs produce competitive results in comparison to other well-established and powerful models such as *CART*, multi-layer perceptron, *XGBoost* or random forests. Third, two datasets containing approximately 1.3 million driving decision examples have been made freely available for academic purposes.

The remaining of the document is organized as follows. Section 2 outlines related work, briefly summarizing recent approaches proposed for decision-making in self-driving cars. Section 3 provides the theoretical background supporting this study. Section 4 describes our testbed simulated environment, the datasets used for evaluation, and model training details. Section 5 analyzes the three tests conducted and their results. Finally, Section 6 highlights conclusions and future work.

2 Related work

The two main architectural paradigms that have emerged in self-driving vehicles are: modular systems and end-to-end driving (Grigorescu et al, 2020). Modular systems decompose the driving task into separate components such as localization, perception, planning, and control. This approach leverages existing expertise in robotics, computer vision, and vehicle dynamics. In contrast, end-to-end driving aims to directly map sensor inputs to driving actions using deep learning. While promising, end-to-end methods currently lack interpretability and safety guarantees for real-world urban driving (Yurtsever et al, 2020).

In (Mechernene et al, 2022), authors evaluate decision trees, random forests (*RFs*), neural networks and fuzzy logic for decision-making in autonomous vehicles. The goal was to devise a transparent, interpretable algorithm mimicking human behavior focusing on lane-changing maneuvers. They found their superiority of DTs, RFs and NNs over fuzzy logic. In (Chen et al, 2018), a hierarchical finite state machine identifies nearby cars and determines safe driving behaviors. In (Huang et al, 2019), a Prolog program is used for autonomous driving by considering 8 neighboring spaces and different road scenarios with static and dynamic entities. (Al-Nuaimi et al, 2021) propose a model checker for multi-agent systems and a probabilistic model checker for formal verification during design and runtime decisions. In (Spanogiannopoulos et al, 2022) authors propose generating real-time, collision-free paths in unknown but near static environments for non-holonomic self-driving cars, using an incremental planner based on rapidly-exploring random trees. The study stated in (Bose et al, 2023) explores challenges in automating road vehicles, with an emphasis on fault-tolerant trajectory tracking. It introduces the differential Lyapunov stochastic and decision fault tree learning for precise and efficient path tracking amidst noise and faults.

The work in (Pini et al, 2023) describes a deep neural network to select the most probable trajectory to ensure safety and comfort. (Fang et al, 2023) present an adversarial learning neural network to evaluate environmental impact, denoise sensor data, and detect anomalies, showing significant improvements over traditional methods. In (Zhang et al, 2023) an object-level explanation approach is introduced, demonstrating heightened persuasiveness compared to pixel-level explanations. Motion prediction systems are vital for safe autonomous vehicle navigation, but Graph Neural Network methods often lack transparency. The Explainable Heterogeneous Graph based Policy model, introduced in (Limeros et al, 2023), enhances interpretability, addressing concerns from users, developers, and regulators and advancing dependable motion prediction systems by improving understanding of traffic dynamics. End-to-end driving models are proposed in autonomous vehicles with an outstanding performance for making motion decisions from pixel-based data (Chib and Singh, 2024).

It also has been shown in the literature of autonomous vehicles that action policies obtained either by solving Markov decision processes (*MDPs*) or by reinforcement learning, are competitive alternatives for behavior selection (Schwartz et al, 2018; Abbas, 2022). For example, in (Song et al, 2023), authors propose a hierarchically structured MDPs for behavior planning, integrating lane keeping, speed control, and collision avoidance. In (Phan-Minh et al, 2023), use inverse reinforcement learning to score and select the best paths based on the learned model. Despite the power of probabilistic logic models for decision-making, to the best of our knowledge, only one proposal has been presented since our initial work to calculate action policies using this programming paradigm (Yang et al, 2023). In that work, the authors elucidate safe reinforcement learning, which aims to learn optimal action policies ensuring safety. They propose probabilistic logic policy gradient which employs probabilistic logic to represent safety constraints as differentiable functions.

3 Preliminaries

3.1 Probabilistic logic factored Markov decision processes

Probabilistic logic factored Markov decision processes are factored Markov decision processes (*fMDPs*) coded in ProbLog syntax (Fierens et al, 2015) to specify and solve infinite-horizon fMDPs. PL-fMDPs are implemented by the MDP-ProbLog package (Bueno, 2017). These models involve an *agent* which decides an action to perform on each *state* of its *environment*, without knowing for sure the resulting state. A positive reward or a negative reward (penalty) is assigned to the agent for each state-action pair. The result of solving a PL-fMDP is a reward ϵ -optimal action policy π_ϵ . This policy associates a suitable action for each state. Optimality is based on the expected cumulative discounted future reward from the current state onward (Sutton and Barto, 1998).

A PL-fMDP program is a tuple $PL = (X_F, \mathcal{A}, \mathcal{U}, \mathcal{T}_r, \mathcal{R}_r, \mathcal{S}, \gamma)$, in which:

- $X_F = \{x_i\}_{i=1}^n$ is a finite, totally ordered set of n *state fluents*¹,
- $\mathcal{A} = \{a_i\}_{i=1}^m$ is a finite, non-empty set of m actions,
- $\mathcal{U}$ is a finite set of utility assignments,
- $\mathcal{T}_r$ is a finite, non-empty set of ProbLog rules called the *transition model*,
- $\mathcal{R}_r$ is a finite set of ProbLog rules called the *reward model*,
- $\mathcal{S}$ is a finite set of supplementary or auxiliary atoms, where $\mathcal{S} \cap X_F = \emptyset$ and $\mathcal{S} \cap \mathcal{A} = \emptyset$,
- $\gamma \in [0, 1)$ is a *discount factor* to upper bound the infinite summation of rewards below infinity.

State fluents in X_F are Boolean state random variables in traditional factored MDPs. Let $\mathbf{X}$ be the set of all possible n -tuples $\mathbf{x} = (x_i)_{i=1}^n$, such that $x_i \in \{1, 0\}$ is the truth value of the state fluent X_i , for all $i = 1, \dots, n$. Thus, a bijective correspondence between $\mathbf{X}$ and the state-space of the environment is assumed. Utilities $(a, u) \in \mathcal{U}$, are ordered pairs, where a is either a state fluent, an action, or a supplementary atom, and $u \in \mathbb{R}$ is the utility value of a . The transition model $\mathcal{T}_r$ is a set of ProbLog rules² that reflect how the truth value of each state fluent evolves given the current state an action. Finally, the reward model $\mathcal{R}_r$ consists on a set of rules that regulates how utilities are assigned to the atoms. Both the state-factored transition probability function and the reward function are key elements to model-based fMDPs which are discussed in the next paragraphs.

The transition probability function $p(\mathbf{x}'|\mathbf{x}, a)$, is a function that governs the state transition dynamics in MDPs, where $\mathbf{x}', \mathbf{x} \in \mathbf{X}$ are the future³ and current state, respectively, and $a \in \mathcal{A}$ is an action. In MDP-ProbLog, the transition probability function is factored according to the next equation:

$$p(\mathbf{x}'|\mathbf{x}, a) = \prod_{i=1}^n p(x'_i = x'_i | \text{obs}(\text{Pa}(\mathbf{x}'_i)), a), \quad (1)$$

¹Here, prime notation is used to distinguish between *post-action* state fluents from *pre-action* state fluents.

²A ProbLog rule such as “ $\alpha::a :- b_1, b_2, \text{not}(b_3)$ ” is interpreted as $p(a|b_1, b_2, \neg b_3) = \alpha$ in probability theory.

³Again we use prime notation to differentiate between a *post-action* state $\mathbf{x}'$, and a *pre-action* state $\mathbf{x}$.

for all $\mathbf{x}' \in \mathbf{X}$, and all pairs of *evidence* $(\mathbf{x}, a) \in \mathbf{X} \times \mathcal{A}$, where $\mathbf{x}'_i \in \mathbf{X}_F$ is the i -th post-action state fluent, $\text{obs}(\text{Pa}(\mathbf{x}'_i))$ is the observed values of the *parent* pre-action state fluents $\text{Pa}(\mathbf{x}'_i) \subseteq \mathbf{X}_F$ of $\mathbf{x}'_i$, that along with the action a influence the probability of $\mathbf{x}'_i$. The term “parent” comes from the Bayesian networks (*BNs*) terminology, a framework frequently used to describe factored transition probability functions (Boutilier et al, 2000). The procedure followed in this work to learn factored transition functions will be revisited later in Section 4.3.2. The probability values $p(\mathbf{x}'_i = 1 | \text{obs}(\text{Pa}(\mathbf{x}'_i)), a)$, for all $i = 1, \dots, n$ are represented by ProbLog rules in $\mathcal{T}_r$. The right side of Eq. 1 is derived by assuming *mutual independence* between post-action state variables given the evidence, and *conditional independence* with some of the pre-action state variables that are not statistically associated to the probability value of $\mathbf{x}'_i$.

In MDP-ProbLog, the reward function is calculated upon utilities in $\mathcal{U}$, as follows:

$$R(\mathbf{x}, a) = \sum_{\substack{(\mathbf{x}, u) \in \mathcal{U} \\ \mathbf{x} \in \mathbf{X}_F}} u p(\mathbf{x} | \mathbf{x}, a) + \sum_{\substack{(\mathbf{a}, u) \in \mathcal{U} \\ \mathbf{a} \in \mathcal{A}}} u p(\mathbf{a} | \mathbf{x}, a) + \sum_{\substack{(\mathbf{s}, u) \in \mathcal{U} \\ \mathbf{s} \in \mathcal{S}}} u p(\mathbf{s} | \mathbf{x}, a), \quad (2)$$

for all $(\mathbf{x}, a) \in \mathbf{X} \times \mathcal{A}$. In Eq. 2, the utility $u \in \mathbb{R}$ associated to an atom $\mathbf{a}$ will be weighted and added up to $R(\mathbf{x}, a)$ as long as $\mathbf{a}$ is consistent with the evidence $(\mathbf{x}, a)$, through the weighting values $p(\cdot | \mathbf{x}, a)$ computed by ProbLog.

3.1.1 Solution of PL-fMDPs

The solution of a PL-fMDP is a reward ϵ -optimal deterministic policy π_ϵ that maximizes a expected cumulative discounted return v_* for every state. This can be done through the *value iteration algorithm*. Value iteration iteratively approximates v_* through the *Bellman update equation*:

$$v_{k+1}(\mathbf{x}) = \max_a [R(\mathbf{x}, a) + \gamma \sum_{\mathbf{x}' \in \mathbf{X}} v_k(\mathbf{x}') \prod_{i=1}^n p(\mathbf{x}'_i = x'_i | \text{obs}(\text{Pa}(\mathbf{x}'_i)), a)], \quad (3)$$

for all $\mathbf{x} \in \mathbf{X}$. In practice, Eq. 3 is repeated until the largest absolute difference $|v_{k+1}(\mathbf{x}) - v_k(\mathbf{x})|$ for any state $\mathbf{x} \in \mathbf{X}$ in two consecutive iterations is lower than the threshold $\frac{\epsilon(1-\gamma)}{2\gamma}$, where $\epsilon > 0$ establishes a maximum error bound, which implies $|v_{k+1}(\mathbf{x}) - v_*(\mathbf{x})| < \frac{\epsilon}{2}$ (Puterman, 2014). Following convergence, the ϵ -optimal policy π_ϵ is retrieved by the equation:

$$\pi_\epsilon(\mathbf{x}) = \arg \max_a [R(\mathbf{x}, a) + \gamma \sum_{\mathbf{x}' \in \mathbf{X}} v_{k+1}(\mathbf{x}') p(\mathbf{x}' | \mathbf{x}, a)], \text{ for all } \mathbf{x} \in \mathbf{X}. \quad (4)$$

3.1.2 MDP-ProbLog program example

We discuss some aspects of the syntax of MDP-ProbLog through a small example of decision-making in a self-driving car. Initially, the self-driving car travels in the right lane of a one-way, two-lane road at a steady speed. As it moves, the self-driving car continuously scans four predefined locations around it to detect other moving obstacle

vehicles. The first location is positioned ahead of the self-driving car in the right lane (denoted as NE), while the other three locations are in the left lane: one in the front-left (NW), one directly to the left (W), and one in the back-left (SW). These locations will be further illustrated in Section 4.1.

Four state fluents indicate whether these zones are occupied or empty. They are named `free_NE`, `free_NW`, `free_W`, and `free_SW`. These variables take the value true (1) if the location is empty and false (0) otherwise. Its main tasks are to drive on the right lane and pass other vehicles on the left, while avoiding rear-end or side-swipe crashes. At each time step, the self-driving car must decide on the most appropriate action among three available alternatives: **cruise**, **change_to_left**, and **keep_distance** accordingly to the presence or not of the surrounding vehicles. The trial ends the first time action **change_to_left** is chosen. The implementation of an MDP-ProbLog code that generates an action policy for this scenario is presented in Listing 1.

Listing 1: MDP-ProbLog example code.

```
% State variables
1 state_fluent(free_NE).
2 state_fluent(free_NW).
3 state_fluent(free_W).
4 state_fluent(free_SW).

% Actions
5 action(cruise).
6 action(keep_distance).
7 action(change_to_left).

% Utilities
8 utility(free_NE(1), 5).
9 utility(free_NW(1), 5).
10 utility(rearEnd_crash(1), -30).
11 utility(sideSwipe_crash(1), -10).
12 utility(keep_distance, -10).
13 utility(change_to_left, -1).

% Reward model
14 0.99::rearEnd_crash(1) :- not(free_NE(1)), cruise, not(keep_distance).
15 0.95::rearEnd_crash(1) :- not(free_NW(1)), change_to_left.
16 0.95::sideSwipe_crash(1) :- not(free_W(1)), change_to_left.

% Transition model
% Conditional probabilities  $p(x'_i = 1 \mid \text{obs}(\text{Pa}(x'_i)), \text{cruise})$ 
17 0.96::free_NE(1) :- free_NE(0), cruise.
18 0.05::free_NE(1) :- not(free_NE(0)), cruise.

% Conditional probabilities  $p(x'_i = 1 \mid \text{obs}(\text{Pa}(x'_i)), \text{change\_to\_left})$ 
19 0.99::free_NW(1) :- free_NW(0), free_W(0), change_to_left.
20 0.05::free_NW(1) :- not(free_NW(0)), change_to_left.
```

```

21 0.05::free_NW(1) :- not (free_W(0)), not (free_SW(0)),
                           change_to_left.

    % Conditional probability  $p(x'_i = 1 \mid \text{obs}(\text{Pa}(x'_i)), \text{keep\_distance})$ 
22 0.1::free_NE(1) :- not (free_NE(0)), keep_distance.

```

We assume the reader has some familiarity with Prolog syntax. For an introduction, we refer to (Sterling and Shapiro, 1994). In MDP-ProbLog, argument 0 identifies pre-action state fluents, while post-action fluents use a value of 1 (primarily in the transition model). Lines 1–4 declare four state fluents, meaning there are $2^4 = 16$ possible states. Lines 5–7 define the three available actions. The utility assignments and reward model are specified in lines 8–16. Specifically, lines 8–9 assign positive utilities to `free_NE` and `free_NW`, rewarding the self-driving car for keeping these spaces free. Lines 10–11 assign negative utilities to complementary atoms representing collisions, which may occur according to the rules in lines 14–16. For instance, line 14 indicates that a rear-end crash is highly probable if the self-driving car chooses **cruise** instead of **keep_distance** while traveling in the right lane. The action **keep_distance** is penalized because the vehicle could become overly cautious, by introducing unnecessary delays. In addition, **change_to_left** receives a negative reward because it is a risky maneuver. The transition model is described in lines 17–22. These rules represent conditional probability values of the form $p(x'_i = 1 \mid \text{obs}(\text{Pa}(x'_i)), a)$ for each action a . For example, in line 22, it is stated that there is only a 10% chance of observing the space ahead in the right lane as empty provided that **keep_distance** is applied and the space is not currently empty.

Table 1 presents the action policy (referred here as “Right”) derived by MDP-ProbLog from this code. In this policy, on the one hand, the self-driving car keeps its distance to the vehicle in front whenever `free_NE`, `free_NW` or `free_SW` are false (0). On the other hand, it selects **cruise** where `free_NE` is true (1). However, it is conservative and changes lanes only when both NW and W are empty.

Note that it is not mandatory to specify rules for all possible reward assignments to state-action pairs or probability values. We believe this is a powerful characteristic of this representation, which can be leveraged by including only partial information, supplemented by experience or expert opinions.

3.2 Machine learning models for comparison

3.2.1 Tree based models

Decision trees (Quinlan, 1986) are perhaps the simplest but at the same time one of the most powerful models in machine learning. They are widely used when having relatively complex datasets. The most commonly used algorithm for learning decision trees is known as CART or classification and regression trees. The CART algorithm (Breiman et al, 1984) can automatically generate data partitions based on the target variable, each with the most homogeneous groupings possible. To measure this homogeneity, the Gini index is used, which measures the degree of “impurity” of a

Table 1: Action policy Right.

Rule #	Occupancy variables				Action
	free_NE	free_NW	free_SW	free_W	
1	0	0	0	0	keep_distance
2	0	1	0	0	keep_distance
3	0	0	1	0	keep_distance
4	0	1	1	0	keep_distance
5	0	0	0	1	keep_distance
6	0	0	1	1	keep_distance
7	1	0	0	0	cruise
8	1	1	0	0	cruise
9	1	0	1	0	cruise
10	1	1	1	0	cruise
11	1	0	0	1	cruise
12	1	1	0	1	cruise
13	1	0	1	1	cruise
14	1	1	1	1	cruise
15	0	1	0	1	change_to_left
16	0	1	1	1	change_to_left

node:

$$Gini = 1 - \sum_{n=1}^C p_i^2, \quad (5)$$

where p_i is the probability of an object being classified to a particular class C .

Gini indices equal to zero indicate pure nodes (that is, with data that belong to a single category), while indices greater than zero and with values up to one indicate nodes with impurities (that is, with data from more than one category). Classification error or entropy are other metrics for selecting the best partition.

In order to identify the best node partition, the CART algorithm defines a cost function that assigns a score to the parent node, using the weighted average of the individual Gini indices of its child nodes. The same procedure is applied iteratively until the data are adequately separated, that is, until the most homogeneous groupings possible are generated. One solution to avoid overfitting is to use hyperparameters such as the maximum depth of the tree.

Decision trees are prone to overfitting on training data. Ensemble methods overcome this issue by combining multiple tree models, such as random forests (Breiman, 2001) or extreme gradient boosting (Chen and Guestrin, 2016). Random forests improve the performance predictions by combining decision trees in parallel. It trains each model from a subset of features randomly chosen without replacement and a subset of samples selected by the bootstrap method (Breiman, 1996). After building multiple decision trees, RFs determines the final prediction by taking the majority vote of class predictions from the models.

Extreme gradient boosting is a scalable and optimized ensemble method for handling sparse data. XGBoost sequentially learns a weighted combination of decision trees using gradient descent. At each iteration, this model tries to build a more robust decision tree by minimizing the prediction errors of the previous model. Each iteration aims to generate a new robust ensemble formed by the newly trained decision tree and

the previous model. The prediction errors of this new ensemble are included in a loss function that should be optimized by subsequent decision tree training. This process is repeated until a stopping criterion is met. The final prediction is the average output of all tree decisions.

3.2.2 Naive Bayes

Naive Bayes is a probabilistic classifier based on the Bayes' theorem that incorporates mutually independence assumptions between input features given the (target) class variable. In particular, given a tuple $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ of n observed features and a set of m classes $\{y_1, y_2, \dots, y_m\}$, the probability of a i -th class y_i is calculated as:

$$\begin{aligned} p(y_i|\mathbf{x}) &= p(y_i|x_1, x_2, \dots, x_n) \\ &= \frac{p(y_i)p(x_1|y_i)p(x_2|y_i) \cdots p(x_n|y_i)}{p(x_1, x_2, \dots, x_n)} \\ &\propto p(y_i)p(x_1|y_i)p(x_2|y_i) \cdots p(x_n|y_i) \\ &\propto p(y_i) \prod_{j=1}^n p(x_j|y_i), \end{aligned} \tag{6}$$

where $p(y_i)$ is the *prior* probability of the class y_i , $p(x_j|y_i)$ is called the *likelihood*, and $\propto$ indicates proportionality. For classification, it is assumed that the class y^* that maximizes $p(y|x_1, x_2, \dots, x_n)$ is the class to which $(x_1, x_2, \dots, x_n)$ belong. For training, the likelihood and prior probabilities can be calculated from a dataset with N samples using *maximum likelihood estimation*: $p(y_i) = \frac{N_i}{N}$ and $p(x_j|y_i) = \frac{N_{jki}}{N_i}$, where N_i represents the number of occurrences of class i , and N_{jki} counts the observations of a feature x_j at its k -th value when the class is y_i (Sucar, 2021). Frequently, additive smoothing is used to smooth counts when certain feature values have zero occurrences for a given class y_i , which could otherwise erroneously result in $p(y_i|x_1, x_2, \dots, x_n) = 0$. In this case, each likelihood is estimated as $p(x_j|y_i) = \frac{N_{jki} + \alpha}{N_i + \alpha p}$, where $\alpha \geq 0$ is a smooth factor.

3.2.3 Multinomial logistic regression

Multinomial logistic regression is a classification method that extends logistic regression for multi-class problems when the features are independent. It considers a set of m classes $\{y_1, y_2, \dots, y_m\}$, where $m > 2$ and a tuple of n observed features $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$. The probability of a class y_i is given by the *softmax* function (Rosa, 2010):

$$\begin{aligned} p(y_i|\mathbf{x}) &= p(y_i|x_1, x_2, \dots, x_n) \\ &= \frac{e^{\beta_{i0} + \beta_{i1}x_1 + \cdots + \beta_{in}x_n}}{\sum_{j=1}^m e^{\beta_{j0} + \beta_{j1}x_1 + \cdots + \beta_{jn}x_n}} \\ &= \frac{e^{\mathbf{x}^T \beta_i}}{\sum_{j=1}^m e^{\mathbf{x}^T \beta_j}}, \end{aligned} \tag{7}$$

for $i = 1, 2, \dots, m$, where $\beta_{i0}, \beta_{i1}, \dots, \beta_{in}$ are *regression coefficients*, β is a matrix for the regression coefficients of each class, and $\sum_{i=1}^m p(y_i|x_1, x_2, \dots, x_n) = 1$. Analogously to naive Bayes, the class y^* that maximizes $p(y_i|x_1, x_2, \dots, x_n)$, for $i = 1, \dots, m$, corresponds to the predicted class of this classifier. The regression coefficients can be estimated using maximum likelihood estimation. The log-likelihood estimation function is:

$$\log L(\beta) = \sum_{i=1}^N \left(\mathbf{x}_i^T \beta_i - \log \sum_{j=1}^m e^{\mathbf{x}_i^T \beta_j} \right), \quad (8)$$

where N is the number of examples available in the dataset and T indicates vector transpose. Equation 8 can be solved through iterative optimization methods, such as gradient-based optimization. Regularization methods, such as Lasso (L1) or Ridge (L2) regression, can be applied during optimization to introduce a penalty value in the log-likelihood function, to help prevent overfitting.

3.2.4 Multi-layer perceptron

A multi-layer perceptron is a *feedforward* neural network with a structure of multiple fully connected layers. The input layer of an MLP is a vector $\mathbf{x} \in \mathbb{R}^p$, where p is the number of input features: $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_p]^T$. On each of its l -th hidden layers, an affine transformation is applied, followed by an activation function α :

$$\mathbf{h}^{(l)} = \alpha(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad (9)$$

where $\mathbf{h}^{(l-1)}$ is the output of the $l - 1$ -th layer $\mathbf{W}^{(l)}$ is the weight matrix, and $\mathbf{b}^{(l)}$ is the bias vector for the l -th layer. Finally, the output layer computes the result of the MLP as:

$$\mathbf{y} = \beta(\mathbf{W}^{(L+1)} \mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}), \quad (10)$$

where $\mathbf{W}^{(L+1)}$ and $\mathbf{b}^{(L+1)}$ are the weight matrix and the bias vector for the output layer, respectively, and β is an activation function (usually softmax for multiclass problems), assuming L hidden layers in the MLP.

For training an MLP, the backpropagation algorithm computes the gradient of the loss function $\mathcal{L}$ with respect to each weight and bias. These parameters are updated using: $\Delta \mathbf{W}^{(l)} = -\eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ $\Delta \mathbf{b}^{(l)} = -\eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$, where $\eta \in (0, 1]$ is the *learning rate*. For inference, the output of the MLP for an input $\mathbf{x}$ is computed as: $\mathbf{y} = f(\mathbf{x}; \Theta)$, where f is a function that represents the MLP and Θ is the set of all its weights and biases.

4 Methodology

4.1 Testbed environment

For data collection and evaluation, we use a self-driving car developed by our team on top of both Webots simulator (Michel, 2004) and the Robot Operating System (ROS) (Stanford Artificial Intelligence Laboratory et al., 2018). These libraries are widely used in robotics as they together constitute a flexible and powerful environment for realistic robot simulation. Webots version is 2022a and ROS version is Noetic.

The system has been tested on Ubuntu 20.04. The testing environment designed on Webots is a self-driving car traveling on a two-lane road with straight segments and curves⁴ and 0, 5 or 10 obstacle vehicles distributed over the road (either static or in motion). Obstacle vehicles are simple Webots car models that incorporate only kinematic properties, but no forces or mass. Their sole task in our setting is to move forward at a steady speed. The main task of the self-driving car is to safely pass all other vehicles while preferring the right lane for advancing and the left lane for overtaking only.

The architecture of the self-driving car includes: i) a perception module, ii) a behavior selection module, and iii) a control module. The perception module detects nearby vehicles, road lanes, and collisions using an accelerometer, a 3D LiDAR, and an RGB video camera. This model also calculates the distance from the self-driving car to the vehicle ahead in the same lane. Figure 2 depicts the surrounding locations where the self-driving car detects other vehicles on the left and right lanes. The behavior selection module comprises a two-level hierarchy of four action policies with which the car selects the action to perform in response to the environment state. The action policy “Selector” (higher level) is aimed to choose between policies Right (presented in Section 3.1.2) or “Left” depending on which lane the self-driving car is in, provided the current lane is known. The selector triggers the policy “Stop” to stop the self-driving car if a collision is detected. The self-driving car selects the next driving maneuver through the policies Left and Right, according to the occupancy of the surrounding locations. The self-driving vehicle considers six “occupancy” Boolean variables called `free_E`, `free_SE`, `free_NE`, `free_NW`, on a straight line `free_SW`, `free_W`. Each one of these variables is true (or 1, in accordance with MLP-ProbLog nomenclature) when a vehicle is detected at the location indicated by its name and it is false (or 0), otherwise. For traveling, the self-driving car can perform fundamentally four actions: (**cruise**, **keep_distance**, **change_to_right**, or **change_to_left**) which are selected in accordance to the presence or absence of other surrounding vehicles. Policy Right was presented in Table 1. Policy Left is shown in Table 2. Finally, the control module determines the speed and steering required for the self-driving car to execute each maneuver.

During operation it is assumed that the vehicles traveling on the left lane moves faster than those on the right lane (8 and 11 km/h, respectively), while the self-driving car is the fastest (its maximum tested speed is near 30 km/h).

4.2 Driving datasets

4.2.1 Autonomous driving dataset

This dataset was constructed through the autonomous operation of the self-driving car in scenarios with a different number of nearby vehicles at random initial positions and distinct speeds. This dataset comprises 46 variables, 893 runs, and 964,087 driving decisions. On each run generates the self-driving car a different traveling path. The total distance traveled is 604.7 km. Real-valued variables include acceleration readings, simulation time, the parameters of the lane borders, (x, y, θ) position of the self-driving

⁴However, in this work, we are considering straight roads only.

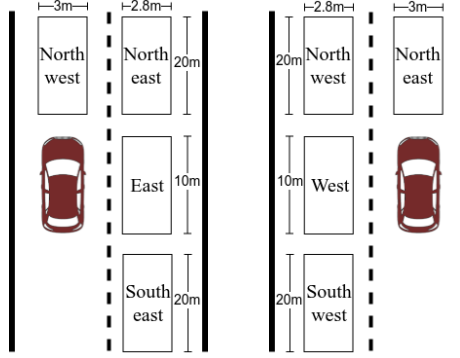


Fig. 2: Preset locations and their labels around the self-driving car (shown in black) where it is expected to see other vehicles in the left lane (left image) and in the right lane (right image). When the vehicle is on the left (resp. right) lane, only the locations Northwest, Northeast, East and Southeast (resp. Northeast, Northwest, West and Southwest) are meaningful.

Table 2: Action policy Left.

Rule #	Occupancy variables				Action
	free_E	free_NW	free_NE	free_SE	
1	0	0	0	0	keep_distance
2	1	0	0	0	keep_distance
3	0	0	1	0	keep_distance
4	0	0	0	1	keep_distance
5	1	0	0	1	keep_distance
6	0	0	1	1	keep_distance
7	0	1	0	0	cruise
8	1	1	0	0	cruise
9	0	1	1	0	cruise
10	0	1	0	1	cruise
11	1	1	0	1	cruise
12	0	1	1	1	cruise
13	1	0	1	0	change_to_right
14	1	1	1	0	change_to_right
15	1	0	1	1	change_to_right
16	1	1	1	1	change_to_right

car throughout the run, (x, y) position of each nearby vehicle, and the distance from the self-driving car to the vehicle ahead in the same lane. At the core of this proposal, the dataset includes the six occupancy Boolean variables described in the previous section. Other Boolean variables relevant to this work are `success`, a flag for collision detection, and `curr_lane`, which is true if the car is in the right lane and false if it is on the left lane. Finally, the dataset also includes a multi-valued variable for the action selected at each step. All runs were performed by the human drivers across $2^4 + 2^4 = 32$ driving scenarios constructed by permuting the presence or absence of

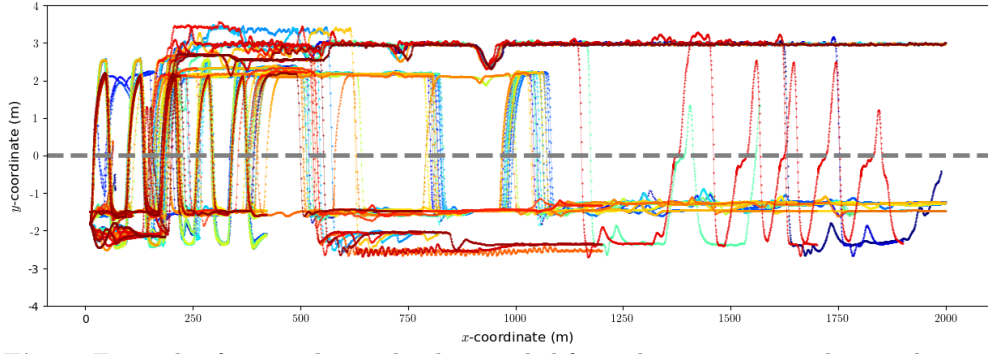


Fig. 3: Example of 120 paths randomly sampled from the autonomous driving dataset (all paths are trimmed to 2 km).

the four obstacle vehicles on each lane at the predetermined positions for obstacle detection. To provide a glimpse of the paths drawn by the self-driving car on each run, Figure 3 shows a random sample of 120 paths in this dataset.

4.2.2 Human drivers dataset

This dataset was collected from the actions selected by four human drivers using a user-friendly, button-based graphical interface. Drivers are second-third year bachelor students, 2 male (Information Technology Program) and 2 female (Mechatronics Program), all within 20 and 23 years old. The human drivers dataset comprises the same variables as the autonomous driving dataset, plus an additional variable to identify each driver. Speed and steering are calculated by the control module. The drivers had some experience with the autonomous operation of the self-driving car, although, apart from using the graphic interface, they were not given any specific instructions on how to drive the vehicle themselves. The dataset contains 3,923 short system runs covering on each run a maximum distance of 200 m, and totaling 331,433 decisions. The total distance traveled is 597.58 km. Driver 1 (female) contributed 32.8% of the database, with 28.2% the Driver 2 (male), Driver 3 (male) with 20.3% and Driver 4 (female) with the rest. Running scenarios are those 32 environments used to collect the autonomous driving dataset. Each run ended when the driver chose the first action to avoid a collision, when a car crash occurred, or when the self-driving car successfully passed all other vehicles. Figure 4 depicts 120 examples collected randomly from this dataset. The only difference with respect to the data collected from the autonomous driving dataset is the addition of a column `driver_id`.

4.3 Model training settings

Training PL-fMDPs, CART, XGBoost, random forests, Naive Bayes, multi-layer perceptron, and logistic regression involves several steps that are explained in the following subsections. First, data preprocessing and feature selection is described.

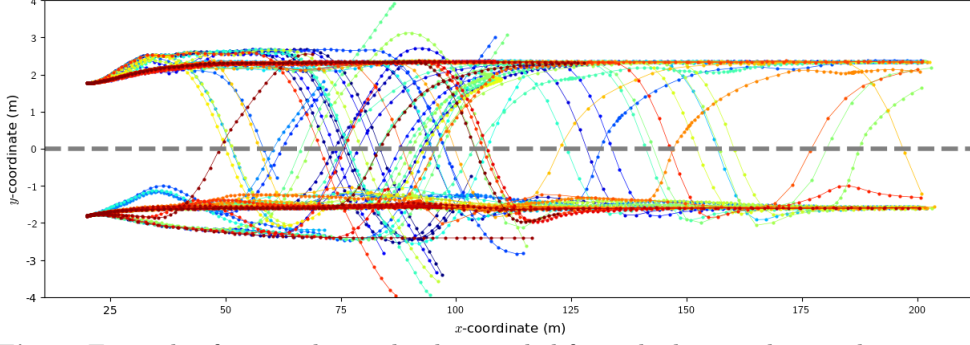


Fig. 4: Example of 120 paths randomly sampled from the human driving dataset and stratified by the driver ID.

4.3.1 Data preprocessing and feature selection

For data preprocessing and feature selection, we remove entries with missing values or those corresponding to car collisions, and selected the six occupancy variables plus the current lane. Let

$$s = (\text{curr_lane}, \text{free_E}, \text{free_SE}, \text{free_NE}, \text{free_NW}, \text{free_SW}, \text{free_W}),$$

be a tuple representing the state of the system. Together with variable *action* the state *s* form a pair (s, action) , where

$$\text{action} \in \{\text{cruise}, \text{keep_distance}, \text{change_to_left}, \text{change_to_right}\}.$$

Instances of this state-action tuple are the basis for training, tuning and testing models under comparison, where the state of the environment is the *input* and the action is the *output*. When the previous procedures are applied to the autonomous driving and the human drivers datasets, they produce two sets, that we call $\mathcal{D}_{\text{auto}}$ and $\mathcal{D}_{\text{humans}}$, respectively. These datasets will be used in subsequent evaluations.

The combination of state variables generates $2^7 = 128$ potential states for the system. By adding the four possible actions the self-driving car can take, the state-action space reaches a size of $2^7 \times 4 = 512$ observable state-action points. One reason to select occupancy variables and current lane is the good results observed with these variables in our previous work. Besides that, due to their binary nature, they can be encoded directly into MDP-ProbLog, currently restricted to Boolean state fluents only. The variable `collision_detection`, originally used to flag collisions, was also omitted from the state *s* because it identifies unusual events (only 24 crashes out of a total of 893 runs from the automated control and 105 crashes out of 3,923 runs from human control). Notwithstanding, this variable is still used for broadcasting emergency stop signals to the system, whenever is needed.

Some frequency statistics of the datasets $\mathcal{D}_{\text{auto}}$ and $\mathcal{D}_{\text{humans}}$ after preprocessing are shown in Table 4. Figure 5 depicts the cumulative frequency distributions of the state-action examples observed in both $\mathcal{D}_{\text{auto}}$ and $\mathcal{D}_{\text{humans}}$ normalized to 1. It can

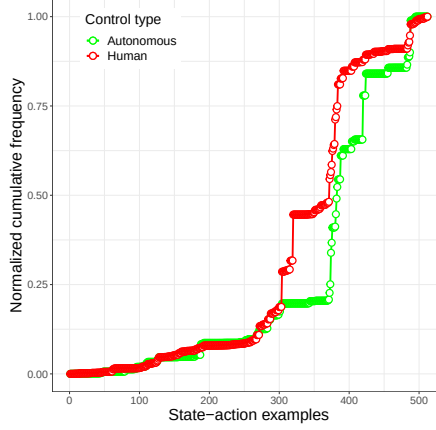


Fig. 5: Normalized cumulative frequency distributions of $\mathcal{D}_{auto}$ and $\mathcal{D}_{humans}$. The 512 unique examples that compose the state-action space were sorted and numerically indexed from 1 to 512.

be seen that both distributions share some similarities and have some differences. A plausible explanation is that there could be more than one safe decision at each state (actions that neither lead to a collision with other vehicles nor cause the car to run off the road). In order to numerically confirm the differences, we use the Jensen-Shannon divergence (Lin, 1991). This is an entropy-based metric that measures distribution shift by contrasting the information content of both distributions. The metric returns a value bounded within $[0, 1]$, where values close to 0 indicate high similarity between distributions, while higher values mean greater dissimilarity. The Jensen-Shannon divergence for our distributions is 0.40. This value indicates that while sharing similarities, both distributions may also exhibit noteworthy differences, primarily because more than one action is usually applicable to a given state. Table 3 counts unique state-action examples stratified by action. these differences justifies the use of the two distributions to assess the robustness of PL-fMDPs with respect to distribution shifts.

Table 3: Number of unique state-action examples stratified by action in $\mathcal{D}_{auto}$ and $\mathcal{D}_{humans}$.

Dataset	Actions				Total by dataset:
	cruise	change_to_left	change_to_right	keep	
$\mathcal{D}_{auto}$	64	33	53	51	201
$\mathcal{D}_{humans}$	86	83	80	64	313

Table 4: Number of actions on each dataset. The entries in “Total by dataset” indicate the size of each dataset.

Action	Autonomous driving	Human drivers	Total by action:
cruise	429,888	237,996	667,884
change_to_right	48,708	13,894	62,602
change_to_left	44,929	15,314	60,243
keep_distance	438,582	62,594	501,176
Total by dataset:	962,107	329,798	1,291,905

4.3.2 Constructing and solving PL-fMDPs

Instead of using the four action policies of the two-level hierarchy described in Section 4.1, we opted to combine the roles of the policy Selector, Left and Right into a single PL-fMDP. The policy Stop was kept separate as an emergency exit when a collision is detected. Finding an optimal state-action policy in PL-fMDPs requires defining the elements that constitute a PL-fMDP. To define a PL-fMDP, we start by learning its factored transition probability function. Here, this task is divided in two steps: (a) identifying the probabilistic dependencies between state fluents via structural learning for BNs and (b) fitting their conditional probability values. As stated in Section 3.1, a factored transition probability function can be represented as a set of Bayesian networks, one Bayesian network for each possible action. Each BN is described graphically through a (two-sliced) DAG. A DAG associated to an action establishes the influence of the pre-action variables over the post-action variables for that action. Hence, it also depicts how the states of the system could evolve after the corresponding action has been executed. To learn the DAGs of an PL-fMDP, data corresponding to the intended action is simply subsetting from $\mathcal{D}_{auto}$. Each DAG is independently estimated using hill-climbing algorithm (Friedman et al, 1999). An example of a DAG is shown in Figure 6. To simplify modeling, typical approaches to factored MDPs focus on describing effects from pre-action fluents to post-action fluents rather than on statistical associations within pre-action fluents or within post-action fluents. Therefore, dependencies within either pre-action or post-action fluents are explicitly forbidden.

A DAG represents the *structure* of the probabilistic dependencies among pre-action and post-action fluents for a given action, outlining dependencies encoded in the conditional probability functions for the factored transition probability function. Once the structure of a DAG is known, the probability values of the conditional probability functions associated with the DAG are fitted by the maximum likelihood estimation criterion (Le Cam, 1990). Because learning PL-fMDPs requires the joint truth values of the pre-action and post-action state fluents, we expand each example $(\mathbf{s}, \mathbf{a})$ available in the dataset, where $\mathbf{s}$ is an instance of the state s of the environment and $\mathbf{a}$ is an action, with the observed post-action state $\mathbf{s}'$. In this manner, data triplets $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ are used to learn BNs. The goal is to represent how the post-action state changes, given that both the current values of the pre-action state fluents and the action currently executed are known. Both learning steps were instrumented using the **bnlearn** package in R programming language (Scutari, 2010).

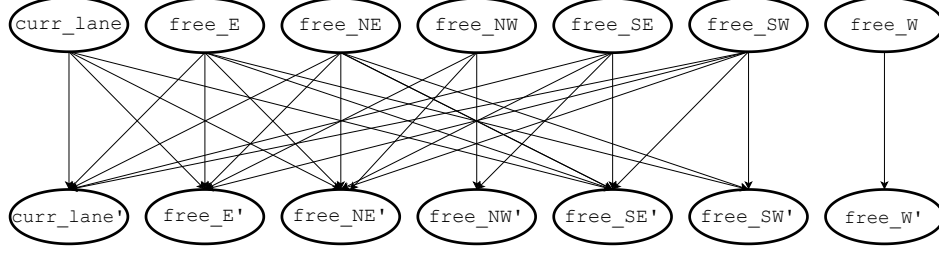


Fig. 6: Example of DAG for the action **change_to_left**. Primed variables indicate post-action state fluents.

The final step in obtaining a PL-fMDP is to define the utility assignments and the reward model, which are specified in Listing 2.

Listing 2: Utilities and reward model for PL-fMDPs.

```
% Utilities
1 utility(move_ahead_right, 20.0).
2 utility(move_ahead_left, 10.0).
3 utility(move_to_right, 30.0).
4 utility(rearEnd_crash, -200.0).
5 utility(sideSwipe_crash, -150.0).
6 utility(run_off_road, -200.0).

% Reward model
9 move_ahead_right :- curr_lane(0), free_NE(0), cruise.
10 move_ahead_left :- not(curr_lane(0)), free_NW(0),
    (not(free_NE(0)); not(free_E(0))), cruise.
11 move_to_right :- not(curr_lane(0)), free_NE(0), free_E(0),
    (free_NW(0); not(free_NW(0))), change_to_right.
12 rearEnd_crash :- not(curr_lane(0)), not(free_NW(0)), cruise.
13 rearEnd_crash :- curr_lane(0), not(free_NE(0)), cruise.
14 rearEnd_crash :- not(curr_lane(0)), not(free_NE(0)),
    change_to_right.
15 rearEnd_crash :- curr_lane(0), not(free_NW(0)), change_to_left.
16 sideSwipe_crash :- not(curr_lane(0)), not(free_E(0)),
    change_to_right.
17 sideSwipe_crash :- curr_lane(0), not(free_W(0)), change_to_left.
18 run_off_road :- not(curr_lane(0)), change_to_left.
19 run_off_road :- curr_lane(0), change_to_right.
```

In Listing 2, positive utilities are assigned in lines 1-2 to motivate the self-driving car to move forward or to overtake by the left side, as defined in the rules 9-10 in the reward model. A positive assignment is also made in line 3 for `move_to_right` to encourage returning to the right lane as soon as possible when circulating in the left lane (line 11). Negative utilities, or penalizations, are assigned in lines 4-6 to three auxiliary atoms: `rearEnd_crash`, `sideSwipe_crash`, and `run_off_road`. The truth conditions in the reward model for these atoms (lines 12-19) detail how collisions

or running off road may occur in our setting. In particular, running off roadway is undesirable, as it may result in the car overturning or causing harm to others. The definition of the utilities and the reward model is similar to our previous work but slightly fine-tuned to integrate the role of policies Selector, Left and Right. These utility assignments and the reward model is kept fixed for all the PL-fMDPs programs constructed for this evaluation. Partial descriptions of state-action pairs in the reward model are sufficient to define how utility values are assigned. In our case, since only some pre-action state fluents are explicitly included in the rules of the reward model, more than one state-action pair will meet the particular condition and will receive the corresponding weighted utility value. For example, due to rule 17, changing to the left lane in any state where the west location is not empty, will receive a penalty. This is because changing lanes in those circumstances would almost surely result in a sideswipe accident. This approach to utility assignment can be convenient for eliciting knowledge from experts about the desirability of some state fluents and actions using a simple and understandable rule-based representation.

The BNs obtained for each action along with the utilities and the reward model presented above, are syntactically translated from **bnlearn** into a PL-fMDP program using a purpose-built R script. The final steps involve calculating the optimal state-action policy for the PL-fMDP program through MDP-ProbLog and creating a corresponding Scikit-learn model (Buitinck et al, 2013) for the resulting state-action policy. We apply MDP-ProbLog default parameters (particularly, with the discount factor $\gamma = 0.90$ and maximum error $\epsilon = 0.1$).

4.3.3 Training machine learning classifiers

The pipeline implemented to evaluate PL-fMDPs alongside CART, XGBoost, RFs, NB, MLP, and LR involves training, hyperparameter tuning, and testing using Scikit-learn. Training and hyperparameter tuning for CART, XGBoost, RFs, NB, MLP, and LR are performed simultaneously using grid search on separate training and tuning datasets. Model instances are created using the training dataset for all possible hyperparameter combinations. The instance achieving the highest weighted average F1 score on an unseen tuning dataset is selected for posterior comparison with PL-fMDPs. Validation on previously unknown data helps reduce overfitting, but in all cases a test dataset different from the training and tuning datasets will be applied for an unbiased evaluation, as discussed in Section 5.

Table 5 outlines the hyperparameters and corresponding values considered for each model. These hyperparameters incorporate critical aspects of model complexity, regularization, and optimization. A brief description of these hyperparameters is presented below:

- Tree-Based Models (CART, XGBoost, Random forests):
 - Maximum Depth (d_{max}): A primary control for model complexity, limiting the depth of individual trees. Mathematically, a tree of depth d has $O(2^d)$ potential leaf nodes. Restricting d_{max} is a key regularization technique to prevent overfitting.

- Number of Estimators (N): For ensemble methods (XGBoost, Random Forest), N dictates the number of individual learners (trees) in the ensemble. Increasing N generally improves performance up to a point, after which diminishing returns and potential overfitting become concerns.
- Minimum Samples to Split Internal Nodes (n_{split}) and Minimum Samples per Leaf Node (n_{leaf}): These parameters, particularly for CART, influence the granularity of decision boundaries. Larger values lead to more conservative splitting, reducing sensitivity to noise.
- Naive Bayes:
 - Alpha (α): Controls the strength of Laplace smoothing, preventing zero probabilities for unseen feature values during training. This is essential for handling previously unobserved data.
- Multi-Layer Perceptron:
 - Hidden Layer Size: Determines the network’s capacity. Larger layers increase model complexity and may improve performance on complex tasks but also raise the risk of overfitting.
 - Activation Function: Introduces non-linearity, crucial for learning complex relationships. ReLU and tanh are common choices, each with different properties regarding gradient propagation and saturation.

The specific choices were guided by the nature of the autonomous driving task and the characteristics of the datasets used in this study.

Note that tuning is applied to all models except PL-fMDPs. As stated in the previous section, for PL-fMDPs, only training and testing are implemented. The discount factor γ and maximum error ϵ are fixed to MDP-Problog’s default values to ensure PL-fMDPs are evaluated under consistent conditions while also stressing them as much as possible, given that other models are tuned using validation data. However, during the overall evaluation all the models are consistently presented with the same training and testing data, and with exception to PL-fMDPs, also with the same tuning data. More details on the data used for training, tuning, and testing are discussed in the next section.

5 Evaluation and results

The details of the test execution for evaluating PL-fMDPs using in-distribution data, as well as samples with distribution shifts and out-of-distribution driving conditions, are described in the following subsections. All experiments were conducted on a desktop computer with an AMD Ryzen 9 5900X 12-core, 24-thread processor, 32 GB of RAM running Ubuntu 20.04.6 LTS Focal Fossa.

Table 5: Hyperparameter space for each machine learning model.

	Scikit-learn hyperparameter	Values	Total number of combinations
CART	Maximum depth	1, 2, 4, 6, 8	180
	Minimum samples to split internal nodes	2, 5, 10, 20, 30, 40	
	Minimum number of samples in a leaf node	1, 2, 5, 10, 15, 20	
XGBoost	Maximum depth	3, 5, 7	36
	Learning rate	0.01, 0.1, 0.2	
	Number of estimators	100, 200	
	Subsample ratio	0.8, 1.0	
RF	Maximum depth	None, 10, 20, 30	36
	Number of estimators	50, 100, 200	
	Maximum number of features for best split	auto, sqrt, log2	
NB	alpha	0.1, 1.0, 10.0	6
	Learning of priors	True, False	
MLP	Size of hidden layers	8, 16, 8×8 , 16×16	32
	Activation function	relu, tanh	
	Solver	adam, sgd	
	Maximum number of iterations	1000, 2000	
LR	Data weight	0.01, 0.1, 1, 10, 100, 1000	6
	Penalty	l2	

5.1 Test 1 - Driving decisions from in-distribution samples

The goal of this test is to assess how well PL-fMDPs learn and predict autonomous driving decisions made by a self-driving car, compared to six supervised learning models in machine learning, leveraging their class prediction capabilities for decision-making.

Here, only the dataset $\mathcal{D}_{auto}$ is used for training, tuning and testing. For evaluation, a custom 5-fold cross-validation split with rotation of test and tune folds is performed as follows. First, unique examples in $\mathcal{D}_{auto}$ are extracted to form a set called $\mathcal{D}_{unique}$. Second, $\mathcal{D}_{unique}$ is shuffled and split randomly into 5 folds stratified by variable *action*. Three of the 5 folds are separated for training, forming a subset is called $\mathcal{D}_{train,u}$. The remaining 2 folds are used for hyperparameter tuning ($\mathcal{D}_{tune,u}$) and testing ($\mathcal{D}_{test,u}$). In this form, 60% of the unique examples go to the training step, 20% to the tuning step and 20% to the testing step. Third, all occurrences of the examples in $\mathcal{D}_{train,u}$, $\mathcal{D}_{tune,u}$, and $\mathcal{D}_{test,u}$ are extracted from $\mathcal{D}_{auto}$ to form three subsets called $\mathcal{D}_{train}$, $\mathcal{D}_{tune}$, and $\mathcal{D}_{test}$, respectively. These three latter subsets are the ones actually used to evaluate each model. Training and tuning are performed as described in Section 4.3.2 and Section 4.3.3 for all the models with datasets $\mathcal{D}_{train}$ and $\mathcal{D}_{tune}$, respectively. Model testing is done with $\mathcal{D}_{test}$. As we decided to average results across all possible partitions of $\mathcal{D}_{unique}$ for training, tuning and testing, the previous process

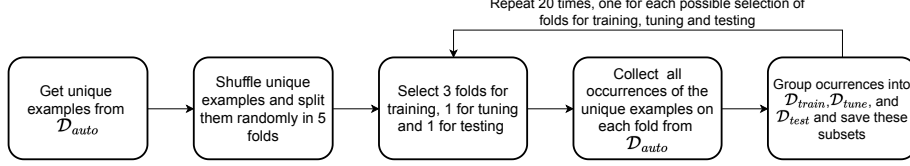


Fig. 7: Sampling procedure to create datasets for training ($\mathcal{D}_{train}$), tuning ($\mathcal{D}_{tune}$) and testing ($\mathcal{D}_{test}$).

is repeated $\binom{5}{3} \times 2! = 10 \times 2 = 20$ times⁵. These sampling steps are depicted in Figure 7.

We chose to split $\mathcal{D}_{unique}$ into 5 folds instead of the more common 10, due to the number of possible combinations involved in creating the evaluation datasets. We believe that a 5-fold partition offers a good balance between evaluation granularity and computational cost. On the one hand, since the selection of $\mathcal{D}_{train,u}$, $\mathcal{D}_{tune,u}$ and $\mathcal{D}_{test,u}$ can be made in 20 ways, the entire evaluation procedure involves 20 repetitions, one repetition for each different selection. In contrast, a 10-fold partition with the same proportions of data for training, tuning, and testing generates $\binom{10}{6} \times \binom{4}{2} \times \binom{2}{2} = 210 \times 6 \times 1 = 1260$ repetitions, which may considerably increase the computational cost of the test.

The procedure has several advantages: i) it accounts for the relatively small size of the state-action space by sampling over unique examples and ensuring the presence of varied examples for training⁶, ii) it promotes generalization by training and evaluating the models with a representative frequency distribution of the unique examples, and iii) it avoids data leakage during training, ensuring that all testing examples in $\mathcal{D}_{test}$ are previously unknown to the models.

In addition to using 100% of the data in $\mathcal{D}_{auto}$, we also evaluate the models using smaller random samples of $\mathcal{D}_{auto}$ stratified by action and comprising 1% and 50% of its data. The objective is to assess the performance of the models when they are presented with datasets of different sizes. Table 6 presents the total number of examples, and the maximum and minimum number of examples included in $\mathcal{D}_{train}$, $\mathcal{D}_{tune}$ and $\mathcal{D}_{test}$ for each data percentage.

5.1.1 Results of Test 1

Precision-recall metrics collected by evaluating the 20 instances of the seven models are presented in Table 7. To interpret both metrics in our setting, assume that most driving decision examples in the dataset $\mathcal{D}_{auto}$ correspond to safe (or successful) actions (that is, actions that do not lead to a car crash). One would expect decision models to reproduce these safe decisions. In this sense, precision reflects the proportion of decisions made by a model that are indeed safe. On the other hand, recall

⁵Exclamation mark “!” indicate factorial and $\binom{5}{3}$ stands for the binomial coefficient. They account for the number of subsets of size 3 that can be extracted from 5 elements, a number doubled due to the two different ways in which the 2 remaining samples can be selected for tuning and testing.

⁶Traditional cross-validation applied directly to $\mathcal{D}_{auto}$ could misrepresent the environment, making the decision-making task overly challenging and potentially introducing data leakage.

Table 6: Sample size and maximum and minimum number of examples used for training, tuning and testing models across the 20 repetitions in Test 1, with respect to the data percentage collected from $\mathcal{D}_{auto}$.

Data	Sample	$\mathcal{D}_{train}$		$\mathcal{D}_{tune}$		$\mathcal{D}_{test}$	
%	size	Maximum	Minimum	Maximum	Minimum	Maximum	Minimum
1%	9,621	7563	4026	3189	918	3189	918
50%	481,053	346033	235092	132640	49391	132640	49391
100%	962,106	652234	507258	227721	135168	227721	135168

indicates how often the decision model predicts an action to be a known safe decision on the dataset (that is, how well its predictions align with the autonomous decisions of the self-driving car, most of which have been validated as safe during operation).⁷ PL-fMDPs have the overall highest performance using these two metrics, only being outperformed by CART in precision and recall at 100%, and also partially surpassed by MLP, XGBoost, RFs in precision at 100%.

Table 7: Mean values for precision-recall measures in Test 1. Numbers are rounded to 6 decimal places.

Models	Mean values					
	1%		50%		100%	
	Precision	Recall	Precision	Recall	Precision	Recall
PL-fMDPs	0.944403	0.938133	0.929732	0.871507	0.913565	0.899488
LR	0.834428	0.757408	0.839850	0.798722	0.876051	0.833854
NB	0.705052	0.740133	0.680724	0.713427	0.770513	0.837179
CART	0.908407	0.879677	0.866737	0.843466	0.949239	0.937162
MLP	0.929938	0.883706	0.925946	0.877589	0.926817	0.878555
RFs	0.900334	0.855579	0.911723	0.808152	0.917636	0.813902
XGBoost	0.934106	0.900030	0.908798	0.870197	0.923129	0.901832

The weighted average F1 score (hereinafter simply called F1 score for short) is the harmonic mean of precision and recall, and therefore provides a balance between both metrics. Figure 8 depicts graphically Tukey’s box plots of the F1 scores obtained when using 1%, 50% and 100% of $\mathcal{D}_{auto}$, respectively, across the 20 instances of all the models. Box plots offer a viewpoint of the frequency distributions of the F1 scores, but a numerical descriptions of these distributions is also useful for comparison. Table 8 summarizes common statistical measurements related to the box plots, as well as the mean and standard deviation of the F1 scores for each model. It is observed that PL-fMDP outperformed the other models when using 1% of the data, achieving superior performance in terms of the median, interquartile range (*IQR*), mean, and standard deviation. However, this comes at the cost of slightly higher skewness and

⁷As stated in Section 4.3.1, dataset $\mathcal{D}_{auto}$ contains only 24 collisions. If one assumes ten decisions preceding a collision (1 second or so) playing an active role in the crash, then there are only $24 \times 10 = 240$ unsafe decisions in the entire dataset.

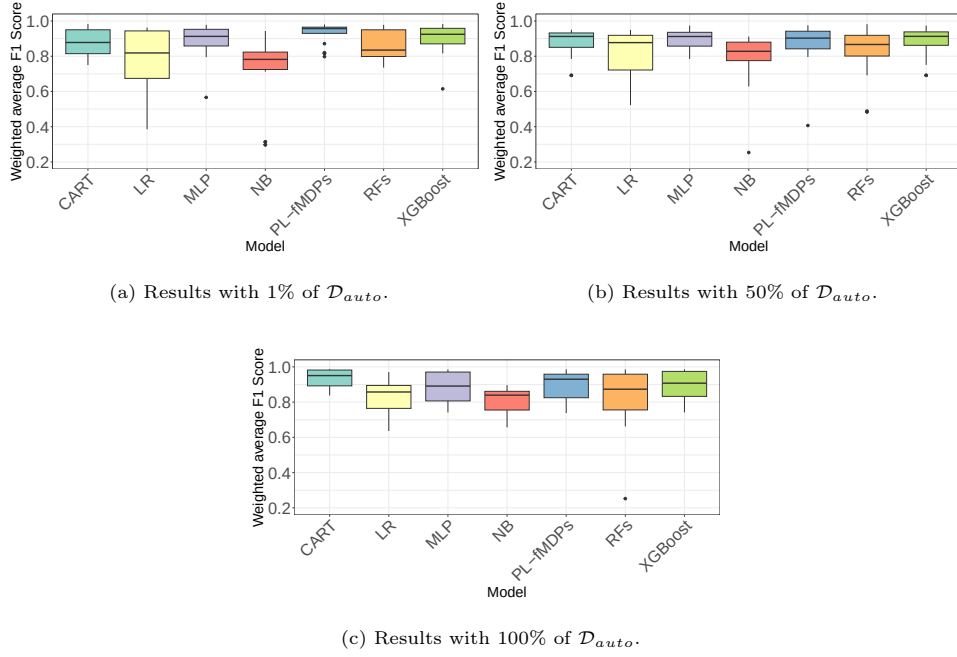


Fig. 8: Weighted average F1 scores obtained in Test 1 for each model using datasets $\mathcal{D}_{train}$, $\mathcal{D}_{tune}$ and $\mathcal{D}_{test}$ for training, tuning and testing, respectively, employing sub-samples of: (a) 1%, (b) 50% and (c) 100% of $\mathcal{D}_{auto}$.

the highest number of outliers. Nevertheless, we do not consider these latter measurements as meaningful, due to the low variance obtained by PL-fMDPs. In addition, PL-fMDPs remain competitive with 50% of $\mathcal{D}_{auto}$, although they exhibit a marginally lower median compared to CART, XGBoost, and MLP. In particular, MLP is the most competitive within this round, obtaining the highest mean F1 score. When using 100% of the data, results of PL-fMDPs are similar to those obtained by MLP and XGBoost, while all three models are outperformed by CART.

From the previous results with F1 scores, it is evident that PL-fMDPs, CART, MLP, and XGBoost exhibit superior overall performance, surpassing LR, NB, and RFs in terms of F1 score. To further investigate these differences, we measured pairwise statistical differences between PL-fMDPs and the other models. First, the Friedman test was applied to detect statistically significant differences among all seven models. The test confirms that such differences exist ($\chi^2(6) = 67.453$, $p\text{-value} = 1.36 \times 10^{-12}$) at a significance level of $\alpha = 0.05$. Table 9 presents the results of a Dunn’s post-hoc test with Bonferroni correction, a non-parametric pairwise comparison based on rank sums, conducted at the same significance level ($\alpha = 0.05$). This analysis indicates that PL-fMDPs differs significantly from NB and LR.

To assess the practical significance of these differences, we calculated effect sizes using Cohen’s d , which quantifies the effect size by scaling the difference in means

between PL-fMDPs and each other model by the pooled standard deviation. We also computed the statistical power of these effect sizes, defined as the probability of detecting a true effect. For 1% of the data, the analysis reveals large effect sizes ($d > 0.8$) in favor of PL-fMDPs compared to NB and LR (for example, for NB, $d = 1.293$ with a power of 0.789). Medium-to-large effect sizes (approximately 0.5 to 0.8) are observed in favor of PL-fMDPs against the other models. However, the statistical power is notably lower in some these latter cases. For instance, with CART, $d = 0.696$ in favor of PL-fMDPs, but the power is only 0.166. For 50% of the data, a meaningful improvement is confirmed only against NB ($d = 0.880$, power = 0.336), while negative differences emerge in favor of MLP ($d = -0.194$, power = 0.007) and XGBoost ($d = -0.040$, power = 0.003). Similar patterns appear with 100% of the data, but now CART ($d = -0.567$, power = 0.088) and XGBoost ($d = -0.098$, power = 0.003) show improvements over PL-fMDPs (negative effect size). Given the low statistical power in many comparisons (below 0.2), particularly with CART, MLP, RFs, and XGBoost, we conclude that additional samples are needed to reliably detect subtle differences involving PL-fMDPs.

The mean and standard deviation of the training time for each model instance, as a function of the data percentage used for training, are shown in Table 10. Since solving PL-fMDPs is actually divided into constructing a PL-fMDP program and calculating its corresponding action policy, time was measured separately. It is shown that CART and NB have the fastest mean training time for each single instance. Constructing and solving PL-fMDPs require considerably more time than training the other models in all cases. However, it should be noted that statistics for PL-fMDPs are not entirely directly comparable with the rest of the models as this calculation does not reflect the time required by tuning to obtain the best instances on each repetition. To show these differences, Table 11 presents the mean time needed to find the best instance of each model. The table also includes the rate of increase in the form of the slope obtained via simple linear regression coefficients. According to this criterion, with the exception of NB, PL-fMDPs exhibit the slowest positive slope, indicating that they scale better than the other models with respect to training sample size. This results should be taken as a grain of salt though, because time required to find the best instances on each one of the 20 repetitions ultimately depends on the number of hyperparameter tuning combinations (see last column of Table 5). As mentioned above, in the case of PL-fMDPs, there are only two hyperparameters, and they do not vary. Nevertheless, slope measures provide a glimpse into how time increases for each model as the sample size grows.

Table 12 shows the mean and standard deviation of the overall testing time across the 20 testing samples. CART, BN, and LR require the least time for testing the samples. APs has the highest testing time in all cases.

Of course, this is not the whole picture. Additional evaluations, such as those conducted in Test 2 and Test 3, are necessary to assess the performance of the models under varying conditions, in particular, under distribution shifts, and when the decision models are deployed in our simulated vehicle.

Table 8: Statistical measurements for the weighted average F1 scores obtained by the models in Test 1 using 1%, 50% and 100% of the dataset $\mathcal{D}_{auto}$.

1% of $\mathcal{D}_{auto}$						
Model	Median	IQR	Skewness	# of outliers	Mean	SD
PL-fMDPs	0.9571702	0.03522644	-1.09608366	5	0.9281295	0.06357865
CART	0.8781678	0.13552391	-0.06877001	0	0.8790806	0.07673837
LR	0.8186449	0.26948825	-0.71796101	0	0.7722582	0.20321537
MLP	0.912808	0.09456024	-1.82198338	1	0.8915769	0.09636618
NB	0.7821024	0.0989189	-1.00753913	4	0.7159961	0.22308011
RFs	0.8349914	0.15160877	0.10426637	0	0.8656478	0.08455625
XGBoost	0.9245873	0.08823295	-1.64910434	1	0.8990021	0.08683047
50% of $\mathcal{D}_{auto}$						
Model	Median	IQR	Skewness	# of outliers	Mean	SD
PL-fMDPs	0.9028539	0.09977559	-2.6198676	1	0.8754878	0.12392001
CART	0.9122772	0.09122693	-2.7195489	3	0.8455549	0.17327237
LR	0.8773905	0.19683234	-0.8736145	0	0.8025706	0.15536629
MLP	0.9121046	0.07844119	-0.5474795	0	0.8944433	0.06067733
NB	0.8086373	0.2269683	-1.1753554	4	0.6892924	0.2724383
RFs	0.8668097	0.11764573	-1.2789513	2	0.8280869	0.14166861
XGBoost	0.9129358	0.07638114	-1.0702268	2	0.8797234	0.08563333
100% of $\mathcal{D}_{auto}$						
Model	Median	IQR	Skewness	# of outliers	Mean	SD
PL-fMDPs	0.9300022	0.1335205	-0.5240466	0	0.8886846	0.08849011
CART	0.9506605	0.08966844	-0.5335305	0	0.9305184	0.05531885
LR	0.8575827	0.13019028	-0.6003364	0	0.831756	0.09679025
MLP	0.8912814	0.16385839	-0.2662245	0	0.8878826	0.08320399
NB	0.8396541	0.10620257	-0.4796223	0	0.7987174	0.08400401
RFs	0.8733461	0.20285136	-1.8588365	1	0.8327084	0.17112759
XGBoost	0.9075687	0.14184215	-0.3510329	0	0.8969449	0.07960681

Table 9: Probability values from Dunn-Bonferroni *post-hoc* test for Test 1 (*p*-values that suggest significant differences are highlighted in boldface red).

Models under comparison	<i>p</i> -value		
	1%	50%	100%
PL-fMDPs - LR	5.54×10^{-03}	0.76	0.76
PL-fMDPs - NB	1.40×10^{-05}	0.004	0.054
PL-fMDPs - CART	0.43	1.0	1.0
PL-fMDPs - MLP	1.0	1.0	1.0
PL-fMDPs - RFs	0.15	1.0	1.0
PL-fMDPs - XGBoost	1.0	1.0	1.0

Table 10: Mean and standard deviation of training time for each model instance relative to the data percentage used in Test 1 (PL-fMDPs refers to the creation of PL-fMDPs programs, and APs denotes the calculation of their action policies).

Model	Training time statistics (in s) per data sample					
	1%		50%		100%	
	Mean	SD	Mean	SD	Mean	SD
PL-fMDPs	2.27	1.30	4.19	0.34	6.36	0.31
APs	1.5	1.46	3.42	0.18	4.2	0.22
CART	0.0013	0.0002	0.04	0.012	0.1	0.02
MLP	2.71	1.21	19.33	7.5	28.7	10.2
LR	0.015	0.005	1.1	0.23	2.4	0.33
NB	0.001	0.0001	0.04	0.009	0.1	0.02
RFs	0.12	0.07	3.35	1.9	7.2	3.9
XGBoost	0.2	0.06	1.9	0.7	3.75	1.43

Table 11: Mean training time for the best instances of the models with respect to the data percentage used in Test 1. The slope of mean training time is calculated using linear regression.

Model	Mean time (in s) per data sample			
	1%	50%	100%	Slope
PL-fMDPs + APs	3.77	8.32	10.56	0.069
CART	0.234	7.2	18	0.180
MLP	86.9	618.56	918.4	8.39
LR	0.09	6.6	14.4	0.145
NB	0.006	0.24	0.6	0.006
RFs	4.32	120.6	259.2	2.58
XGBoost	7.2	68.4	135	1.29

Table 12: Mean and standard deviation of testing time for each model and set of data in Test 1.

Model	Testing time statistics (in s) per data sample					
	1%		50%		100%	
	Mean	SD	Mean	SD	Mean	SD
APs	0.05	0.02	2.5	0.72	5.15	0.91
CART	0.0006	0.0001	0.003	0.001	0.0055	0.0015
MLP	0.001	0.0009	0.045	0.03	0.11	0.07
LR	0.0007	0.0004	0.005	0.002	0.01	0.003
NB	0.0005	0.00003	0.004	0.0013	0.0084	0.003
RFs	0.01	0.009	0.24	0.2	0.37	0.052
XGBoost	0.005	0.0035	0.03	0.011	0.07	0.024

5.2 Test 2 - Testing with distribution shift

The primary goal of this test is to evaluate the agreement of action policies obtained from PL-fMDPs when presented with examples collected from similar driving scenarios but involving a different decision maker. We are not particularly interested in personalization of driving decisions as a means of mimicking human driving style or preferences, for example as proposed in (Yang et al, 2025). Instead, we focus on assessing the robustness of PL-fMDPs in handling moderate distribution shifts compared to other machine learning models. This analysis could be useful to develop strategies for interaction modeling or collision avoidance, where predicting the driving decisions of surrounding vehicles may be necessary. Besides, distribution shift is a common concern in machine learning applications within real life, constantly changing operational environments.

We use the 20 models instances constructed in Test 1 for each model as the basis for this test. However, this time testing data comes from $\mathcal{D}_{humans}$. The main steps for constructing the testing samples consist of shuffling $\mathcal{D}_{humans}$, then randomly dividing the samples into 20 testing folds of equal size stratified by `action` and `driver_id`, while removing the latter variable for testing. We also evaluate with two random samples taking 1% and 50% of $\mathcal{D}_{humans}$, respectively, stratified again by `action` and `driver_id`.

Table 13: Sample size and the range of human control examples used for testing models across 20 repetitions in Test 2, as a function of the testing data percentage from $\mathcal{D}_{humans}$.

Data	Sample	Number of examples from $\mathcal{D}_{humans}$	
%	size	Maximum	Minimum
1%	3,298	172	156
50%	164,898	8,252	8,237
100%	329,798	16,496	16,483

5.2.1 Results of Test 2

Precision and recall metrics are introduced in Table 14 for each model. It shows a decline in performance for all models in both precision and recall under distribution shift introduced for testing. Despite this, PL-fMDPs maintains stable performance, exhibiting less variability in recall than most models and therefore consistently learning known safe actions in the training datasets. However, it is slightly outperformed in precision by CART, RFs, XGBoost, and MLP across training sizes, suggesting that these models would decide safe actions more effectively. Overall, PL-fMDPs remains a reliable model with consistent results.

Figure 9 graphically depicts the average and standard deviation of F1 scores for each model including the scores of all drivers, as a function of the sample percentage of $\mathcal{D}_{humans}$ used for testing. The gold standard for evaluation under distribution shifts is

Table 14: Mean values for precision-recall measures in Test 2. Numbers are rounded to 6 decimal places.

Models	Mean values					
	1%		50%		100%	
	Precision	Recall	Precision	Recall	Precision	Recall
PL-fMDPs	0.803114	0.678619	0.776017	0.646259	0.767824	0.655076
LR	0.734330	0.630672	0.734839	0.555425	0.747885	0.600309
NB	0.663296	0.665791	0.664696	0.660048	0.686952	0.704245
CART	0.783900	0.664319	0.777990	0.621438	0.805104	0.674353
MLP	0.799149	0.629187	0.790264	0.568503	0.792664	0.583592
RFs	0.782268	0.585594	0.791063	0.539689	0.793991	0.561954
XGBoost	0.795238	0.698195	0.793031	0.606856	0.798516	0.658255

to calculate the difference in performance between in-distribution data and distribution shifts. Table 15 shows these differences. The results suggest that there is indeed a considerable distribution shift that all models struggle to handle. In particular, CART, MLP, RF, XGBoost and PL-fMDPs, which are the most competitive models in Test 1, also exhibit the most pronounced drops in performance. Notably, MLP shows the largest overall decline. We acknowledge that a deeper analysis is required to identify the nature of distribution shift between $\mathcal{D}_{auto}$ and $\mathcal{D}_{humans}$, for example, by analyzing the contribution of each input variable or action preferences, although this particular study is beyond the scope of this paper. However, from these results it is still not straightforward to discern performance differences between PL-fMDPs and the other models. To measure statistical differences in Test 2, we apply statistical tests to F1 scores. According to the Friedman test, significant differences exist ($\chi^2(6) = 47.044$, $p\text{-value} = 1.834 \times 10^{-08}$) at a significance level of $\alpha = 0.05$. Table 16 summarizes the probability values from the Dunn-Bonferroni test. As in Test 1, PL-fMDPs do not show significant differences compared to CART, MLP, and XGBoost. Cohen’s d and power analysis were computed for Test 2 results. For 1% of the data, large effect sizes favor PL-fMDPs over NB ($d = 1.588$, power = 0.953) and RFs ($d = 1.200$, power = 0.701). Moderate to large effects occur in favor of PL-fMDPs with respect to CART ($d = 0.286$, power = 0.014), LR ($d = 0.688$, power = 0.160), and MLP ($d = 0.661$, power = 0.142). As in Test 1, statistical power is often low, except for NB and RFs. A negative difference favors XGBoost ($d = -0.174$, power = 0.006). For 50% of the data, better performance is observed for PL-fMDPs against LR ($d = 1.342$, power = 0.828), MLP ($d = 0.989$, power = 0.461), NB ($d = 0.815$, power = 0.269), and RFs ($d = 1.456$, power = 0.900), with moderate to large effects. No significant differences are found for CART ($d = 0.259$, power = 0.012) and XGBoost ($d = 0.285$, power = 0.014). With 100% of the data, PL-fMDPs outperform RFs ($d = 1.295$, power = 0.790), LR ($d = 0.613$, power = 0.112) and MLP ($d = 0.869$, power = 0.324), while CART ($d = -0.919$, power = 0.379), NB ($d = -0.061$, power = 0.003), and XGBoost ($d = -0.562$, power = 0.086) have the edge on PL-fMDPs. Low power still limits detection of small differences, suggesting again larger sample sizes could clarify these findings.

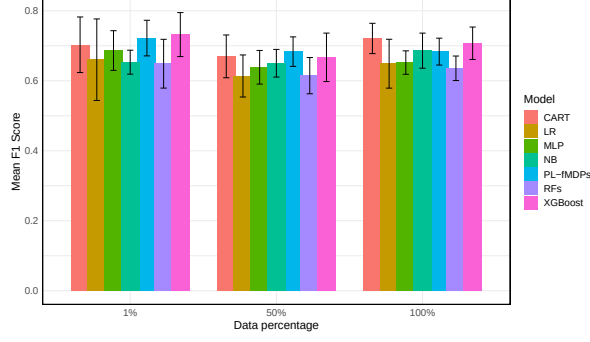


Fig. 9: Mean and standard deviation of weighted average F1 scores for each model across 20 repetitions of the test, as a function of the sample percentage of $\mathcal{D}_{humans}$ used for testing in Test 2.

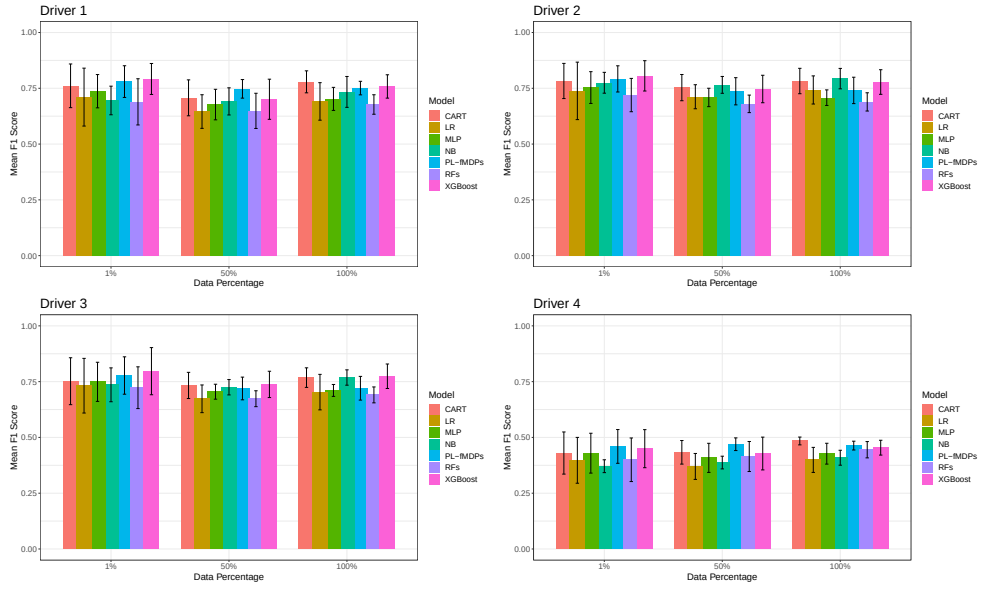


Fig. 10: Mean and standard deviation of weighted average F1 scores for each model and driver, as a function of the sample percentage of $\mathcal{D}_{humans}$.

Figure 10 depicts the weighted average F1 scores again, but this time disaggregated for each driver. This figure reveals important discrepancies between the driving decisions of different drivers (particularly Driver 4) and the decisions made by the trained models. As stated above, identifying the factors contributing to these differences or developing strategies for the models to adapt to them remains an area for further investigation in our setting.

Table 15: Differences in weighted average F1 scores obtained by the models between Test 1 and Test 2.

1% sample			
Model	Mean (Test 1)	Mean (Test 2)	Difference
PL-fMDPs	0.9281295	0.7220326	0.2060969
CART	0.8790806	0.7029710	0.1761096
LR	0.7722582	0.6602238	0.1120344
MLP	0.8915769	0.6864286	0.2051483
NB	0.7159961	0.6531948	0.0628013
RFs	0.8656478	0.6487897	0.2168581
XGBoost	0.8990021	0.7319982	0.1670039

50% sample			
Model	Mean (Test 1)	Mean (Test 2)	Difference
PL-fMDPs	0.8754878	0.6833558	0.192132
CART	0.8455549	0.6697658	0.1757891
LR	0.8025706	0.6137044	0.1888662
MLP	0.8944433	0.6386435	0.2557998
NB	0.6892924	0.6500828	0.0392096
RFs	0.8280869	0.6146378	0.2134491
XGBoost	0.8797234	0.6670331	0.2126903

100% sample			
Model	Mean (Test 1)	Mean (Test 2)	Difference
PL-fMDPs	0.8886846	0.6833437	0.2053409
CART	0.9305184	0.7209535	0.2095649
LR	0.831756	0.6488248	0.1829312
MLP	0.8878826	0.6520427	0.2358399
NB	0.7987174	0.6860632	0.1126542
RFs	0.8327084	0.6357431	0.1969653
XGBoost	0.8969449	0.7072524	0.1896925

Table 16: Probability values from Dunn-Bonferroni *post-hoc* test using $\mathcal{D}_{auto}$ for training and tuning and $\mathcal{D}_{humans}$ for testing in Test 2.

Models under comparison	<i>p</i> -value		
	1% of $\mathcal{D}_{humans}$	50% of $\mathcal{D}_{humans}$	100% of $\mathcal{D}_{humans}$
PL-fMDPs - LR	0.22	0.003905134	0.84
PL-fMDPs - NB	0.002447193	0.89	1.0
PL-fMDPs - CART	1.0	1.0	0.50
PL-fMDPs - MLP	0.49	0.16	0.31
PL-fMDPs - RFs	0.005	0.002	0.024
PL-fMDPs - XGBoost	1.0	1.0	1.0

5.3 Test 3 - Autonomous control on unknown driving scenarios

In this test, the self-driving car is challenged by placing it in driving situations that are almost certain to cause collisions. We evaluate what we consider salient models in

the previous tests, namely, PL-fMDPs, CART, MLP, and XGBoost to construct four different behavior decision modules using Scikit-learn. The environment comprises a two-lane straight road and up to three simple Webots vehicles traveling forward in a straight line. The test assess our self-driving car in three unknown operational environments: (a) overtaking an obstacle vehicle in the right lane while other vehicles rapidly approach from behind in the left lane, (b) passing on the left while another vehicle approaches from the opposite direction in the left lane, and (c) passing through a crossroads while an obstacle vehicle traveling transversely fails to yield the right-of-way at the intersection. Figure 11 illustrates these three scenarios.

All three scenarios involve significant changes in the speed and trajectories of the obstacle vehicles compared to those originally captured in datasets $\mathcal{D}_{auto}$ and $\mathcal{D}_{humans}$. Their positions and speeds also lead to common types of accidents: sideswipe, head-on, and T-bone (side-impact) crashes, respectively. For example, in scenario (a) shown in Figure 11a, the distances of both the vehicle ahead in the right lane and the vehicles behind in the left lane might encourage the self-driving car to pass by switching to the left lane. However, if the fast-approaching obstacle vehicles in the left lane are not detected in time, a sideswipe crash could occur. In scenario (b), depicted in Figure 11b, the presence of a vehicle ahead (in yellow) approaching from the opposite direction is unexpected, potentially leading to a head-on collision. In scenario (c) (Figure 11c), a non-stopping obstacle vehicle follows a perpendicular trajectory, increasing the likelihood of a T-bone accident. The initial positions of the obstacle vehicles were predetermined. However, before each repetition, all vehicles, including the self-driving car, were randomly shifted forward or backward by ± 2 m, following a uniform probability distribution.

We randomly selected the 11th instance of PL-fMDPs, CART, MLP, and XGBoost trained with 100% of the dataset $\mathcal{D}_{auto}$. Each scenario and model underwent 30 decision trials, resulting in a total of $30 \times 3 \times 4 = 360$ runs for comparison.

5.3.1 Results of Test 3

The evaluation is based on the number of runs in which the self-driving car successfully evaded obstacle vehicles and continued on its path. Table 17 summarizes the number of successful runs for each scenario and decision module as well as the total number of successful runs for each module. In all cases, a severe performance decline is evident, though this is not surprising since the scenarios were unknown to the self-driving car and were intentionally designed to be more challenging than the training cases. PL-fMDPs performed almost twice as well as the best of the other models. However, this result is still less than acceptable, achieving a trial success rate of $\frac{29}{90} \approx 0.32$. This increase could be influenced by the competitive recall performance of PL-fMDPs, which appears to prioritize safe actions. Most of the failures occurred due to collisions between cars and, to a lesser extent, the self-driving car running off the road. In scenario (a), the majority of accidents were sideswipe collisions between the self-driving car and the blue vehicle. In scenario (b), the self-driving car most often was hit in a head-on collision with the yellow vehicle, and occasionally with vehicles in the right lane, which prevented it from merging back to the right. In scenario (c), the self-driving

car consistently approached the crossing vehicle in a hazardous manner, leading to side collisions or actions that caused it to run off the road.

Regarding time performance, the difference in average testing time between PL-fMDPs and the other models observed in Test 1 suggests a performance degradation for the former during deployment. However, this difference was negligible because the ROS loop rate was set to 100 ms, which exceeds the combined latency from both the action selection process and the internal time required for the self-driving car to execute an action.

Table 17: Number of successful runs for each behavior selection module, with 30 repetitions for each model in each driving scenario (90 in total for each model).

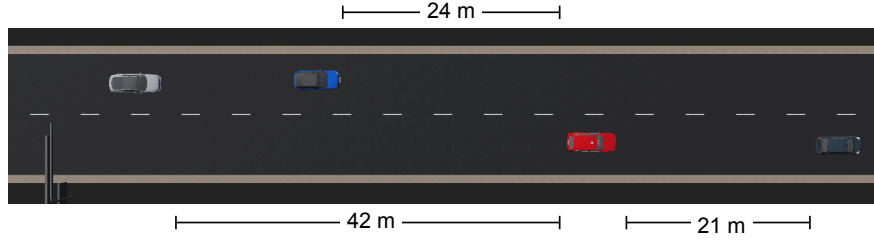
Scenario	Number of runs per model			
	PL-fMDPs	CART	MLP	XGBoost
(a) Passing maneuver with fast vehicles behind	2	0	1	1
(b) Obstacle vehicle in opposite direction	12	7	2	6
(c) Obstacle vehicle in transverse direction	15	8	7	6
Successful runs by module:	29	15	10	13

5.4 Discussion

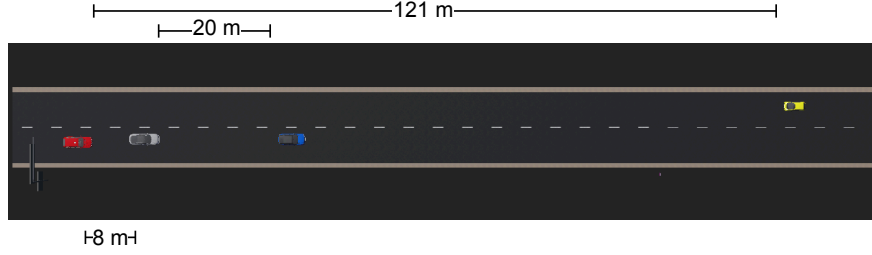
Results obtained with PL-fMDPs with respect to CART, XGBoost, RFs, NB, MLP, and LR are fairly consistent across tests and demonstrate the competitive strengths of the former for decision-making in our evaluation settings.

It is shown that, when considering in-distribution data for testing (Test 1), PL-fMDPs clearly outperform the other models, particularly with a small-sized dataset (that is, at 1% of the data), in terms of precision-recall and F1 score. PL-fMDPs are outdone by MLP in recall at 50% and by MLP, RFs, XGBoost, and CART in precision-recall at 100%. Although a small decay in performance is observed for PL-fMDPs as the percentage of input data increases, they still provide competitive overall performance. Robustness to small data or distribution shift can also be explored in other critical domains, such as medicine, where exhaustive datasets are frequently difficult to collect. Recall all models except PL-fMDPs were tuned. Yet, its rule-based design is still competitive enough. This could represent an advantage, for example, when constructing models under time pressure. In contrast, both the learning time for constructing each instance of PL-fMDPs and testing time are the highest. PL-fMDPs require learning each edge that relates pre- and post-action variables in the BNs using all available data. We believe, however, that the if-then rules of the PL-fMDPs’ action policies can be notably reduced by merging or eliminating redundant comparisons.

When testing with distribution shift (Test 2), all models suffered a considerable performance drop (for example, 0.255 at 50% for MLP, the worst case). On the one hand, this drop confirms the distribution shift between the state-action samples of $\mathcal{D}_{auto}$ and $\mathcal{D}_{humans}$. CART, MLP, XGBoost, and RFs often exceed PL-fMDPs in precision at 50% and 100%, especially CART at 100% (0.805 vs. 0.768). Power analysis



(a) Passing an obstacle vehicle on the left lane with fast-moving vehicles approaching from behind. Vehicles on the left move at a maximum speed of 25 km/h and the leading vehicle on the right at 14 km/h.



(b) Facing a vehicle in opposite direction while passing on the left lane. Yellow vehicle in opposite direction moves at a maximum speed of 25 km/h and leading vehicles on the right at 14 km/h.



(c) Meeting a vehicle traveling in transverse direction. Obstacle vehicle travels at a maximum speed of 25 km/h.

Fig. 11: Unknown scenarios for Test 3. The task for the self-driving car (in bright red) is to evade obstacle vehicles. The maximum speed of the self-driving car is 20 km/h.

in Test 1 and Test 2 indicate that inspecting small differences among these models requires increasing the sample size of F1 scores. However, results also show robust, consistent performance of PL-fMDPs across all percentages of testing data.

Finally, the evaluation of four self-driving cars (Test 3), each equipped with a different decision model (PL-fMDPs, CART, MLP, and XGBoost) and tested on three highly difficult scenarios, shows better (but not satisfactory) performance of PL-fMDPs compared to the other models. A potential explanation for this result lies in the learning methods applied in each case. While “pure” supervised learning is used for CART, MLP, RFs, and XGBoost, their performance can be affected when the examples for learning are imbalanced or do not include all possible instances of the data. However, the combination of supervised learning for BNs, utility-based expert knowledge, maximization of future rewards, and explicit uncertainty handling in PL-fMDPs may increase their robustness to unseen scenarios, as seen in Test 2 stable recall (for example, 0.655 at 100%). Various alternatives can be explored to improve results in Test 3. For example, one could add adaptation techniques or more variables, such as speed or distance, to fine-grain the state-action space and achieve more detailed descriptions. Unfortunately, this last option reveals a key limitation of the current implementation of PL-fMDPs: state fluents are restricted to Boolean variables only, which coarsens descriptions of the state-action space that could limit detection of subtle differences between models. Probabilistic logic offers alternatives too, such as annotated disjunctions (Vennekens et al, 2004), to include multi-valued variables. Another plausible alternative that does not involve re-implementing a behavior decision model is to incorporate specialized obstacle-avoidance decision components. Notwithstanding, our main goal is to explore the competitiveness of PL-fMDPs in decision-making within our setting, an objective we believe has been supported through the results obtained across this study.

6 Conclusions and future work

We presented an extensive evaluation of the effectiveness of probabilistic logic factored Markov decision processes (PL-fMDPs) in modeling behavior selection for self-driving cars in three different scenarios. The study has demonstrated the capability of PL-fMDPs to achieve competitive performance in autonomous vehicle behavior selection, particularly within controlled simulation environments. These results suggest that PL-fMDPs hold promise for interpretable decision-making in autonomous systems. In addition, we successfully created and made publicly available two comprehensive data sets that cover more than 1.3 million driving decisions, offering valuable resources to the research community.

Recognizing limitations, specifically sensitivity to distribution shift, simplified state/action space representations, and computational complexity, future work will focus on expanding PL-fMDPs expressiveness to incorporate richer contextual information like dynamic traffic and intricate road geometries. Crucially, we will prioritize the development of optimized inference algorithms tailored for real-time deployment in resource-constrained embedded systems. In the middle term, we plan to incorporate complementary mechanisms based on causal discovery to improve car performance, evaluate the ideas presented here in the real world using scale models of autonomous vehicles, and relax the problem domain using decentralized planning using PL-fMDPs

in a collaboration context. Addressing these challenges will advance the development of safer, more reliable, and transparent autonomous driving systems.

Acknowledgments

The authors thank Gloria-de-la-Garza, Omar Pérez, Sergio Vázquez, and Melchor Hernández for their support in conducting experiments during the early versions of this document.

Declarations

- Funding This work was partially supported by UNAM-DGAPA under grant IT102424 and AI Consortium - CIMAT-CONAHCYT.
- Conflict of interest/Competing interests The authors have no competing interests to declare that are relevant to the content of this article.
- Data availability The autonomous driving dataset and the human drivers dataset can be found at: <https://www.kaggle.com/autonomousvehicle> under Open Database License (ODbL) v1.0.
- Materials availability Two videos showing the operation of self-driving car can be found at: <https://github.com/hector-aviles/AIRJ2024>.
- Code availability The source code of the self-driving vehicle can be found at: <https://github.com/hector-aviles/AIRJ2024> under CC0-1.0 license.

References

- Abbas Q (2022) E-share: Autonomous share public transit using IoT-enabled AI technologies. *International Journal of Computer Science and Network Security* 22(6):716–726. <https://doi.org/10.22937/IJCSNS.2022.22.6.90>
- Aksjonov A, Kyrki V (2022) A safety-critical decision making and control framework combining machine learning and rule-based algorithms. *arXiv preprint arXiv:220112819*
- Al-Nuaimi M, Wibowo S, Qu H, et al (2021) Hybrid verification technique for decision-making of self-driving vehicles. *Journal of Sensor and Actuator Networks* 10(3). <https://doi.org/10.3390/jsan10030042>
- Avilés H, Negrete M, Machucho R, et al (2022) Probabilistic logic markov decision processes for modeling driving behaviors in self-driving cars. In: *Ibero-American Conference on Artificial Intelligence*, Springer, pp 366–377, https://doi.org/10.1007/978-3-031-22419-5_31
- Avilés H, Negrete M, Reyes A, et al (2024) Autonomous Behavior Selection For Self-driving Cars Using Probabilistic Logic Factored Markov Decision Processes. *Applied Artificial Intelligence* 38(1):e2304942 (25 pages). <https://doi.org/10.1080/08839514.2024.2304942>

- Badue C, Guidolini R, Carneiro RV, et al (2021) Self-driving cars: A survey. *Expert Systems with Applications* 165:113816. <https://doi.org/https://doi.org/10.1016/j.eswa.2020.113816>
- Bose SSC, Alfurhood BS, Gururaj HL, et al (2023) Decision fault tree learning and differential lyapunov optimal control for path tracking. *Entropy* 25(3). <https://doi.org/10.3390/e25030443>
- Boutilier C, Dearden R, Goldszmidt M (2000) Stochastic dynamic programming with factored representations. *Artificial intelligence* 121(1-2):49–107. [https://doi.org/https://doi.org/10.1016/S0004-3702\(00\)00033-3](https://doi.org/https://doi.org/10.1016/S0004-3702(00)00033-3)
- Breiman L (1996) Bagging predictors. *Machine learning* 24:123–140. <https://doi.org/https://doi.org/10.1007/BF00058655>
- Breiman L (2001) Random forests. *Machine learning* 45:5–32. <https://doi.org/https://doi.org/10.1023/A:1010933404324>
- Breiman L, Friedman J, Stone C, et al (1984) *Classification and Regression Trees*. Taylor & Francis, <https://doi.org/https://doi.org/10.1201/9781315139470>
- Bueno T (2017) MDP-ProbLog. URL <https://github.com/thiagopbueno/mdp-problog>
- Bueno TP, Mauá DD, De Barros LN, et al (2016) Markov decision processes specified by probabilistic logic programming: representation and solution. In: 2016 5th Brazilian Conference on Intelligent Systems (BRACIS), IEEE, pp 337–342, <https://doi.org/10.1109/BRACIS.2016.068>
- Buitinck L, Louppe G, Blondel M, et al (2013) Api design for machine learning software: experiences from the scikit-learn project. *arXiv preprint arXiv:13090238* <https://arxiv.org/abs/1309.0238>, [arXiv:1309.0238](https://arxiv.org/abs/1309.0238)
- Carvalho DV, Pereira EM, Cardoso JS (2019) Machine learning interpretability: A survey on methods and metrics. *Electronics* 8(8):832. <https://doi.org/https://www.mdpi.com/2079-9292/8/8/832>
- Chavira M, Darwiche A (2008) On probabilistic inference by weighted model counting. *Artificial Intelligence* 172(6-7):772–799. <https://doi.org/https://doi.org/10.1016/j.artint.2007.11.002>
- Chen K, Yang B, Pei X, et al (2018) Hierarchical control strategy towards safe driving of autonomous vehicles. *Journal of Intelligent & Fuzzy Systems* 34(4):2197–2212. <https://doi.org/10.3233/jifs-171186>
- Chen S, Hu X, Zhao J, et al (2024) A review of decision-making and planning for autonomous vehicles in intersection environments. *World Electric Vehicle Journal* 15(3). <https://doi.org/10.3390/wevj15030099>

- Chen T, Guestrin C (2016) XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, pp 785–794, <https://doi.org/https://doi.org/10.1145/2939672.2939785>
- Chib PS, Singh P (2024) Recent advancements in end-to-end autonomous driving using deep learning: A survey. *IEEE Transactions on Intelligent Vehicles* 9(1):103–118. <https://doi.org/10.1109/TIV.2023.3318070>
- Chougule A, Chamola V, Sam A, et al (2024) A comprehensive review on limitations of autonomous driving and its impact on accidents and collisions. *IEEE Open Journal of Vehicular Technology* 5:142–161. <https://doi.org/10.1109/OJVT.2023.3335180>
- Da Lio M, Cherubini A, Papini GPR, et al (2023) Complex self-driving behaviors emerging from affordance competition in layered control architectures. *Cognitive Systems Research* 79:4–14. <https://doi.org/https://doi.org/10.1016/j.cogsys.2022.12.007>
- Derkinderen V, Manhaeve R, Zuidberg Dos Martires P, et al (2024) Semirings for probabilistic and neuro-symbolic logic programming. *International Journal of Approximate Reasoning* 171:109130. <https://doi.org/https://doi.org/10.1016/j.ijar.2024.109130>
- Eiter T, Hecher M, Kiesel R (2024) aspmc: New frontiers of algebraic answer set counting. *Artificial Intelligence* 330:104109. <https://doi.org/https://doi.org/10.1016/j.artint.2024.104109>
- Fang Y, Min H, Wu X, et al (2023) Toward interpretability in fault diagnosis for autonomous vehicles: Interpretation of sensor data anomalies. *IEEE Sensors Journal* 23(5):5014–5027. <https://doi.org/10.1109/JSEN.2023.3236838>
- Fierens D, Van den Broeck G, Renkens J, et al (2015) Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming* 15(3):358–401. <https://doi.org/https://doi.org/10.1017/S1471068414000076>
- Friedman N, Nachman I, Peér D (1999) Learning Bayesian network structure from massive datasets: the “sparse candidate” algorithm. In: Proceedings of the Fifteenth conference on Uncertainty in artificial intelligence, pp 206–215
- Ghraizi D, Talj R, Francis C (2023) An overview of decision-making in autonomous vehicles. *IFAC-PapersOnLine* 56(2):10971–10983. <https://doi.org/https://doi.org/10.1016/j.ifacol.2023.10.793>, 22nd IFAC World Congress
- Grigorescu S, Trasnea B, Cocias T, et al (2020) A survey of deep learning techniques for autonomous driving. *J Field Robot* 37(3):362–386. <https://doi.org/10.1002/rob.21918>

- Hu J, Wang Y, Cheng S, et al (2025) A survey of decision-making and planning methods for self-driving vehicles. *Frontiers in Neurorobotics* 19(145192). <https://doi.org/10.3389/fnbot.2025.145192>
- Huang L, Liang H, Yu B, et al (2019) Ontology-based driving scene modeling, situation assessment and decision making for autonomous vehicles. In: 2019 4th Asia-Pacific Conference on Intelligent Robot Systems (ACIRS), IEEE, pp 57–62, <https://doi.org/10.1109/ACIRS.2019.8935984>
- Le Cam L (1990) Maximum likelihood: An introduction. *International Statistical Review / Revue Internationale de Statistique* 58(2):153–171. <https://doi.org/https://doi.org/10.2307/1403464>
- Limeros SC, Majchrowska S, Johnander J, et al (2023) Towards explainable motion prediction using heterogeneous graph representations. *Transportation Research Part C: Emerging Technologies* 157. <https://doi.org/10.1016/j.trc.2023.104405>
- Lin J (1991) Divergence measures based on the shannon entropy. *IEEE Transactions on Information theory* 37(1):145–151. <https://doi.org/https://doi.org/10.1109/18.61115>
- Mechernene A, Judalet V, Chaïbet A, et al (2022) Detection and risk analysis with lane-changing decision algorithms for autonomous vehicles. *Sensors* 22(21). <https://doi.org/10.3390/s22218148>
- Michel O (2004) Cyberbotics Ltd. WebotsTM: Professional mobile robot simulation. *Journal of Advanced Robotics Systems* 1(1):39–42. <https://doi.org/https://doi.org/10.5772/5618>
- Niu Y, Wang Y, Xiao M, et al (2025) Reliable safety decision-making for autonomous vehicles: a safety assurance reinforcement learning. *Transportmetrica B: Transport Dynamics* 13(1):2439997. <https://doi.org/https://doi.org/10.1080/21680566.2024.2439997>
- Phan-Minh T, Howington F, Chu TS, et al (2023) Driveirl: Drive in real life with inverse reinforcement learning. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, pp 1544–1550, <https://doi.org/10.1109/ICRA48891.2023.10160449>
- Pini S, Perone CS, Ahuja A, et al (2023) Safe real-world autonomous driving by learning to predict and plan with a mixture of experts. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, pp 10069–10075, <https://doi.org/10.1109/ICRA48891.2023.10160992>
- Puterman ML (2014) Markov decision processes: discrete stochastic dynamic programming. John Wiley & Sons, [https://doi.org/https://doi.org/10.1016/S0927-0507\(05\)80172-0](https://doi.org/https://doi.org/10.1016/S0927-0507(05)80172-0)

- Quinlan J (1986) Induction of decision trees. *Machine Learning* 1(1):81–106. <https://doi.org/10.1007/BF00116251>
- Riguzzi F (2023) *Foundations of probabilistic logic programming: Languages, semantics, inference and learning* (2nd ed.). River Publishers, <https://doi.org/http://dx.doi.org/10.1201/9781003427421>
- Rosa GJM (2010) The elements of statistical learning: Data mining, inference, and prediction by hastie, t., tibshirani, r., and friedman, j. *Biometrics* 66(4):1315–1315. <https://doi.org/10.1111/j.1541-0420.2010.01516.x>
- Schwarting W, Alonso-Mora J, Rus D (2018) Planning and decision-making for autonomous vehicles. *Annual Review of Control, Robotics, and Autonomous Systems* 1:187–210. <https://doi.org/10.1146/annurev-control-060117-105157>
- Scutari M (2010) Learning Bayesian Networks with the bnlearn R Package. *Journal of Statistical Software* 35(3):1–22. <https://doi.org/10.18637/jss.v035.i03>
- Song Y, Han S, Huh K (2023) A self-driving decision making with reachable path analysis and interaction-aware speed profiling. *IEEE Access* 11:122302–122314. <https://doi.org/10.1109/ACCESS.2023.3328628>
- Spanogiannopoulos S, Zweiri Y, Seneviratne L (2022) Sampling-based non-holonomic path generation for self-driving cars. *Journal of Intelligent & Robotic Systems* 104(1). <https://doi.org/10.1007/s10846-021-01440-z>
- Stanford Artificial Intelligence Laboratory et al. (2018) Robotic operating system. URL <https://www.ros.org>
- Sterling L, Shapiro EY (1994) *The art of Prolog: advanced programming techniques*. MIT press, <https://doi.org/https://dl.acm.org/doi/10.1145/960114.29664>
- Sucar LE (2021) *Probabilistic graphical models*, 2nd edn. Springer Cham, <https://doi.org/https://doi.org/10.1007/978-3-030-61943-5>
- Sutton R, Barto A (1998) Reinforcement learning: An introduction. *IEEE Transactions on Neural Networks* 9(5):1054–1054. <https://doi.org/10.1109/TNN.1998.712192>
- Tampuu A, Matiisen T, Semikin M, et al (2020) A survey of end-to-end driving: Architectures and training methods. *IEEE Transactions on Neural Networks and Learning Systems* 33(4):1364–1384. <https://doi.org/https://doi.org/10.1109/TNNLS.2020.3043505>
- Vennekens J, Verbaeten S, Bruynooghe M (2004) Logic programs with annotated disjunctions. In: *International Conference on Logic Programming*, Springer, pp 431–445, https://doi.org/https://doi.org/10.1007/978-3-540-27775-0_30

- Yang H, Zhou Y, Wu J, et al (2025) Human-guided continual learning for personalized decision-making of autonomous driving. *IEEE Transactions on Intelligent Transportation Systems* pp 1–0. <https://doi.org/10.1109/TITS.2024.3524609>
- Yang WC, Marra G, Rens G, et al (2023) Safe reinforcement learning via probabilistic logic shields. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI*, pp 5739–5749, <https://www.ijcai.org/proceedings/2023/0637.pdf>
- Yurtsever E, Lambert J, Carballo A, et al (2020) A survey of autonomous driving: Common practices and emerging technologies. *IEEE Access* 8:58443–58469. <https://doi.org/10.1109/ACCESS.2020.2983149>
- Zhang C, Deguchi D, Okafuji Y, et al (2023) More persuasive explanation method for end-to-end driving models. *IEEE Access* 11:4270–4282. <https://doi.org/10.1109/ACCESS.2023.3235739>
- Şensoy M, Kaplan LM, Julier S, et al (2025) Risk-aware classification via uncertainty quantification. *Expert Systems with Applications* 265:125906. <https://doi.org/https://doi.org/10.1016/j.eswa.2024.125906>

Authors and Affiliations

Héctor Avilés¹, Verónica Rodríguez², Alberto Reyes³, Rubén Machucho¹, Marco Negrete⁵, Gabriel Ramirez⁴, Karelly Rivera¹, Myriam Pequeño¹, Ingridh Gracia¹, Nicolás Luévano¹

✉Marco Negrete
marco.negrete@ingenieria.unam.edu

Héctor Avilés
havilesa@upv.edu.mx

Verónica Rodríguez
veromix@mixteco.utm.mx

Alberto Reyes
areyes@ineel.mx

Rubén Machucho
rmachuchoc@upv.edu.mx

Gabriel Ramirez
grtorres@cinvestav.mx

Karelly Rivera
1930435@upv.edu.mx

Myriam Pequeño
2330006@upv.edu.mx

Ingridh Gracia
2330319@upv.edu.mx

Nicolás Luévano
2330215@upv.edu.mx

¹*Polytechnic University of Victoria, 87138 Tamaulipas, Mexico*

²*Technological University of the Mixteca, 69004 Oaxaca, Mexico*

³*National Institute of Electricity and Clean Energy, 62490 Cuernavaca, Mexico*

⁴*Center for Research and Advanced Studies of the National Polytechnic Institute, Tamaulipas Campus, 87138 Tamaulipas, Mexico*

⁵*National Autonomous University of Mexico, 04510 Mexico City, Mexico*

Corresponding author:

Marco Negrete
e-mail: marco.negrete@ingenieria.unam.edu